

HAETAE-2 Jasmin 구현 검증

수학자와 암호이론가를 위한 현재 증명 전선

HAETAE top-down EasyCrypt 작업 문서

상태 기준: 2026년 8월 10일

Abstract

이 문서는 HAETAE-2의 논문 명세, Jasmin 구현, EasyCrypt 증명 사이에서 현재 실제로 연결된 부분을 수학적 명제로 재구성한다. 핵심 성과는 모든 종료하는 내부 KeyGen 실행이 반환하는 비밀키의 첫 992바이트가 검증키와 같다는 부분정확성 정리와, 이 불변식으로부터 Sign과 Verify의 실제 생성 절차가 원시/내부(raw/internal) 경로에서 같은 32바이트 메시지 다이제스트 접두부를 만들고 같은 도전값 접미부(challenge suffix) 상태에 도달한다는 관계적 정리이다. 이 국소 경계에서는 통계적 손실이 0이다. 실제 원시 KeyGen 호출자의 순차 VK/SK 내보내기, 정수/64비트 주소 바인딩, Sign/Verify의 정확 추적(exact trace), 성공한 Verify의 꼬리 구간(tail) 도달 및 해시 입력 바인딩도 검증되었다. 실제 Sign 출력의 프레임 조건(frame condition), 구간 국소(region-local) 해시 동치, 실제 Sign 다음 Verify의 순차 추적까지 담겼다. 다만 완료된 추적의 μ 를 운반하는 생성/재전달(product/replay) 간선은 명시적 논문 경계로 동결되어 있다.

서명 기능 경로에서는 정준(canonical) 입력 아래 1056바이트 접두부 코덱의 왕복(round trip), HBZ 준비/적용(prepare/apply), 실제 표 인증서(table certificate), 순수 rANS 단계와 정규화 바이트 스택(normalization-byte stack), 실제 접미부 복사와 검증 결합 모듈(harness)의 제어를 증명했다. Week 10은 배열과 목록의 접미부 연결, 실제 W32/W64 단어 단계, 생성된 내부 정규화 반복문의 바이트/프레임 보존과 커서 무언더플로, 최종 4바이트 직렬화의 앞 정리, 그리고 실제 `_rans_encode`의 성공 시 크기 범위 $4 \leq 1024 - \text{off} \leq 1024$ 를 추가했다. Week 11은 꼬리 인식 불변식(tail-aware invariant), 생성 단어 단계 연결(generated word-step bridge), 한 기호 외부 전이와 최종 직렬화 합성(serialization composition)을 실제 생성 절차 안에 설치했다. 그 결과 종료한 실제 `_rans_encode`가 실패를 반환하거나, 성공 시 전체 반환 접미부가 순수 `trace_bytes`와 같고 오프셋-길이 등식 및 초기 접두부 프레임을 만족한다는 성공 조건 부분정확성(success-conditioned partial correctness) 정리가 컴파일된다. Week 12는 실제 생성 `_rans_decode`의 초기 4바이트 파싱, 구체적 표 조회와 단어 갱신, 0/1/2바이트 정규화 재생(normalization replay), 외부/내부 반복 불변식과 최종 거절 검사를 직접 담았다. 그 결과 정확한 `trace_bytes` 입력을 받아 종료한 실행이 1024개 기호를 복원하고, 정확한 크기를 소비하며, 출력 꼬리를 보존하고 `bad = 0`을 반환한다는 정확 추적 부분정확성(exact-trace partial correctness)이 컴파일된다. Week 13은 실제 인코더의 접미부와 실제 복사의 점별 구간 등식을 디코더의 정확 추적 입력으로 운반하고, 동일한 결합 모듈에서 실패 시 디코더 미실행과 성공 시 1024기호 복원을 함께 증명했다. 67개 작성 검증 대상(authored proof target)의 전체 검증도 통과했으며 운영 판정은 rANS 핵심 역함수 관문 통과를 뜻하는 GO-CORE다. 다만 실제 성공 증인(success witness), 인코더/디코더 종료와 실제 전체 HBZ 래퍼 합성은 여전히 열려 있다. 따라서 $\delta_{\mu, \text{raw_api_accept}} = 0$ 과 $\delta_{\text{Encoding}} = 0$ 은 아직 주장할 수 없다. 접미부 코덱, 문맥, 하이비트(highbits)/LSB, 전체 도전값, 재시도 분포와 구현 수준 EUF-CMA도 열려 있다. 이 문서는 이 경계를 분리해 읽기 위한 전용 안내서이다.

한 문장 요약

현재 증명은

$$\text{prefix}(sk) = vk \Rightarrow \text{take}_{992}(\mu_{\text{Sign}}^{64}) = \mu_{\text{Verify}}^{32} \Rightarrow \text{도전값 접미부 상태 일치}$$

를 실제 생성된 원시/내부 절차에 대해 담았고, 실제 원시 API의 내보내기, 정확 추적, Sign 출력 프레임 및 구간 국소 메모리 연결까지 올라갔다. 기능 경로는 실제 서명

접두부 코덱에 이어 HBZ 준비/적용, 실제 표 인증서와 순수 rANS 단계, 공통 정규화 바이트 추적, 실제 접미부 복사와 3회 호출 결합의 제어를 단았다. Week 10에는 실제 인코더 내부 쓰기 의미와 성공 시 크기 범위까지 추가되었다. Week 11은 순수 꼬리를 보존하는 정확한 내부 불변식, 생성 단어 갱신, 외부 반복과 마지막 네 저장을 합성하여 실제 인코더의 성공 접미부 정제를 단았다. Week 12는 같은 순수 추적을 실제 디코더가 앞에서부터 소비하여 원래 기호, 정확한 소비 크기와 출력 꼬리 프레임을 복원함을 단았다. Week 13은 실제 인코더-복사-디코더 세 절차를 직접 호출하는 결합 모듈에서 실패/성공 분기와 성공 시 전체 기호 복원을 합성했다. 다만 완료된 추적의 저장된 μ 를 연결하는 생성/재전달 정리는 **DEFERRED**이고, 실제 인코더 성공, 인코더/디코더 종료, 실제 전체 HBZ/h 접미부 및 Verify 관문이 남아 있다. 따라서 API/인코딩 무손실 (zero loss)과 EUF-CMA로의 승격은 아직 허용되지 않는다.

목차

1 문서의 독자, 범위, 상태 언어	4
1.1 무엇을 읽기 쉽게 만들려는가	4
1.2 한글(영어) 병기 원칙	4
1.3 상태 용어	4
1.4 스냅샷과 대상	5
2 논문 수준의 수학적 대상	5
2.1 HAETAЕ-2의 최소 배경	5
2.2 mode-2 서명 코덱(signature codec)의 기능정확성 목표	6
2.3 HBZ 잎(leaf), rANS 단일 단계와 바이트 추적(byte trace)의 수학적 구조	6
2.4 Week 10 실제 인코더의 수학적 경계	8
2.5 Week 11 실제 인코더 추적 폐쇄	8
2.6 Week 12 실제 디코더 추적 폐쇄	9
2.7 Week 13 실제 rANS 핵심 합성 폐쇄	10
2.8 최상위 기능정확성 목표	10
2.9 최상위 보안 합성 목표	10
3 명제를 읽기 위한 형식 언어(EasyCrypt)	11
3.1 핵심 술어와 관계	11
3.2 Hoare 명제와 관계적 동치	14
3.3 정확 추적(exact trace)과 승인 꼬리(accepted tail)	14
3.4 배열 크기와 논리적 길이	15
4 현재 닫힌 증명 사슬	15
4.1 1단계: KeyGen이 반환하는 패킹된 키(packed-key) 접두부	15
4.2 2단계: 로컬 배열에서 Verify 메모리로	16
4.3 3단계: 복사 보조절차(copy helper)에서 실제 원시 KeyGen 호출자까지	16
4.4 4단계: Sign/Verify의 실제 원시(raw) μ 절차	17
4.5 5단계: 도전값 접미부(challenge suffix)의 동일성	17
4.6 6단계: 실제 원시 Sign/Verify 호출자의 정확 추적(exact trace)	18
4.7 7단계: 승인 경로 생성 어댑터(accepted-path generated adapter)와 남은 간극	18
4.8 8단계: Sign 출력 프레임(Sign-output frame)과 구간 국소 순차 추적(region-local sequential trace)	18
4.9 9단계: 실제 mode-2 서명 접두부 코덱(signature prefix codec)	19
4.10 10단계: 실제 HBZ 잎(leaf), 표 인증서(table certificate)와 순수 rANS 단계	19

4.11	11단계: rANS 바이트 스택(byte stack), 실제 접미부 복사와 결합 제어(control harness)	20
4.12	12단계: 실제 인코더 내부 의미와 성공 시 크기	21
4.13	13단계: Week 11 실제 인코더 추적 폐쇄	22
4.14	14단계: Week 12 실제 디코더 의미 정제	24
4.15	15단계: Week 13 실제 rANS 핵심 역함수 합성	25
5	보안 관점에서 정확히 무엇을 얻었는가	26
5.1	국소 무손실(zero loss)의 의미	26
5.2	현재 주장 행렬	26
5.3	가정, 계약, 증명 의무의 구분	29
6	남은 증명 의무와 권장 진행 순서	29
6.1	Week 5/6이 닫은 실제 API 경계	29
6.2	동결된 보안 관문(gate): 저장 μ 의 생성/재전달(product/replay)	29
6.3	최근 닫힌 관문과 현재 진행 관문: 실제 rANS 핵심에서 전체 HBZ로	30
6.4	분리된 기능정확성 경로	30
6.5	그 다음 관문: 문맥(context)과 전체 도전값 전사(challenge transcript)	31
6.6	그 이후의 기능 및 확률적 의무	31
6.7	의존성에 따른 작업 순서	32
7	신뢰 경계와 재현 방법	33
7.1	무엇을 신뢰하는가	33
7.2	증명 검증 파이프라인	33
7.3	이 문서 빌드	34
A	정리와 소스 인덱스	35
B	기호 및 약어	41

1 문서의 독자, 범위, 상태 언어

1.1 무엇을 읽기 쉽게 만들려는가

이 작업공간의 기존 latex/ 문서는 주차별 연구개발 기록이다. 반면 이 문서는 수학자와 암호이론가가 다음 세 질문에 바로 답할 수 있도록 정리 순서를 바꾼다.

1. 논문 속 어떤 수학적 등식이 구현의 어떤 바이트 불변식으로 내려오는가?
2. 현재 EasyCrypt가 실제로 검증한 가장 강한 정리는 무엇이며, 그 전제는 무엇인가?
3. 그 정리가 전체 기능정확성이나 EUF-CMA 정리가 되려면 무엇이 더 필요한가?

본문의 사실 우선순위는 실제 EasyCrypt 선언, CLAIM_LEDGER.md, 추출 매니페스트, 최신 주차별 보고서 순이다. 주차별 문서의 앞부분은 해당 시점의 역사적 스냅샷이므로 현재 상태 판정에는 다음 자료를 우선한다.

- WEEK13_REPORT.md, Week 13 연구 메모 latex/sections/29-week13-rans-core-composition.tex, 독립 검증 기록 logs/week13-independent-verifier.md;
- PRODUCT_REPLAY_DECISION.md, SIGNATURE_CODEC_OBLIGATIONS.md, MODE2_HBZ_CODEC_OBLIGATIONS.md;
- RANS_PROOF_DESIGN.md, RANS_BYTE_STACK_INVARIANT.md, RANS_ENCODER_INVARIANT.md, RANS_DECODER_INVARIANT.md와 운영 문서.

줄 번호는 편집에 따라 변하므로 본문은 파일 경로와 선언 이름을 안정적인 식별자로 쓴다.

1.2 한글(영어) 병기 원칙

이 문서는 수학자와 암호이론가가 구현 용어를 원문과 대응시킬 수 있도록 다음 규칙을 사용한다.

1. 기술 용어가 처음 나올 때는 한글을 먼저 쓰고 영어 원어를 괄호에 둔다. 예를 들어 정제(refinement), 불변식(invariant), 추적(trace), 프레임 조건(frame condition), 접두부(prefix), 접미부(suffix)처럼 쓴다.
2. 같은 문맥에서 반복할 때는 한글을 우선한다. 다만 코드와 직접 대조하는 문장에서는 이미 병기한 영어를 다시 쓸 수 있다.
3. EasyCrypt의 정리명, 절차명, 주장 식별자, 파일명과 코드 조각은 정확한 검색을 위해 번역하지 않는다. rANS, HBZ, W32, W64, EUF-CMA 같은 표준 약어도 그대로 둔다.
4. “actual”은 실제 생성 절차를 뜻할 때 “실제(actual)”, “canonical”은 “정준(canonical)”, “success-conditioned”는 “성공 조건부(success-conditioned)”로 처음 병기한다.

1.3 상태 용어

PROVED	표시된 전제 아래의 EasyCrypt 정리가 새 캐시 없이 컴파일된다. 부모 목표의 전제가 모두 닫혔다는 뜻은 아니다.
PARTIAL	부모 목표에 필요한 진짜 하위 정리는 있으나, 부모 목표 전체에는 미해결 전제나 호출 경계가 남아 있다.
SPECIFIED	목표의 타입과 논리식은 컴파일되지만 이를 증명하는 정리는 아직 없다.
BLOCKED	필요한 추출, 메모리 의미론, 확률분포, 수치오차, 또는 암호학적 환원이 구체적으로 빠져 있어 다음 합성을 진행할 수 없다.
DEFERRED	필요한 간극은 정확히 식별됐지만 현재 허용된 증명 규칙과 범위에서는 닫히지 않아 명시적 논문 경계로 동결했다. 증명되었거나 불필요하다는 뜻이 아니다.

경계 명제 1.1 (PROVED의 해석). 이 문서에서 **PROVED**는 항상 “표시된 전제를 만족하는 실행에 대한 해당 정리가 검증되었다”는 뜻이다. 특히 Hoare 부분정확성 정리의 **PROVED**는 종료, losslessness, 출력분포의 일치를 함의하지 않는다.

1.4 스냅샷과 대상

표 1: 이 문서가 고정하는 검증 경계	
항목	고정된 대상
논문 명세	CryptoLab_HAETAE.pdf, 46쪽, 2026-02-04 생성, SHA-256 80d28b3af9af2a27dba0eb4746f9b575 5e05cb9a86bb6596f45fda06c605f3f6 [2]
파라미터 집합	HAETAE-2만: $n = 256$, $(k, \ell) = (2, 4)$, $q = 64513$, $\eta = 1$, $\tau = 58$
구현	고정된 haetae-ref-jasmin/ 소스와 jasmin2ec 생성물 [3]
작성 증명	haetae-topdown-easycrypt/easycrypt/ 아래 매니페스트된 67개 EasyCrypt 대상의 완전 검증 기준선
최신 판정	GO-CORE; 실제 인코더와 디코더 정제 및 실제 인코더-복사-디코더 핵심 역함수는 성공 조건부 Hoare 부분정확성으로 PROVED , 다음 관문은 실제 전체 HBZ 래퍼 합성
재사용 증명	고정된 KeyGen, NTT, sampler, singularity, FFT 결과를 원래 전제 그대로 가져오는 import-only 어댑터 [1]
제외 범위	다른 파라미터 집합, 물리적 부채널, fault, RNG 품질, 그리고 EUF-CMA로부터 자동으로 따라오지 않는 메모리 안전성 및 SCT

경계 명제 1.2 (67대상 검증의 해석). WEEK13_REPORT.md가 기록한 Week 13 완료 실행의 종료 표식은 RESULT PASS authored-targets=67 cache=no-eco다. 이는 매니페스트, 추출 재 생성, 증명 구멍/공리(proof-hole/axiom) 검사, 67개 대상의 새 컴파일, 기준선과 주차별 LaTeX 빌드가 통과했다는 뜻이다. 실행 중 덮어쓰는 logs/verify-all-summary.txt는 마지막 RESULT PASS가 있을 때만 완료 증거다. Hoare 정리의 **PROVED**는 여전히 실제 절차의 종료나 성공 분기 도달 가능성을 뜻하지 않는다.

이 문서가 주장하지 않는 것

완전한 KeyGen 분포 일치, 전체 Sign/Verify 기능정확성, 공용 문맥 추적 (public-context transcript) 일치, 승인된 원시 API 추적의 직접 μ 등식, 전체 1474바이트 서명 코덱, 실제 rANS 인코더의 성공 도달, 인코더/디코더 종료, 실제 전체 HBZ 래퍼의 결합 왕복, 인코딩 무손실, 또는 Jasmin 구현의 종단간 EUF-CMA 보안은 아직 증명되었다고 주장하지 않는다.

2 논문 수준의 수학적 대상

2.1 HAETAE-2의 최소 배경

논문은

$$R = \mathbb{Z}[X]/(X^{256} + 1), \quad R_q = \mathbb{Z}_q[X]/(X^{256} + 1), \quad q = 64513$$

위의 모듈 격자 서명으로 HAETAE를 기술한다 [2]. 모드(mode) 2에서는 $(k, \ell) = (2, 4)$, 작은 비밀계수 범위는 $\eta = 1$, 이진 도전값(binary challenge)의 무게는 $\tau = 58$ 이다. 검증키는 $vk = (seedA, b_1)$ 로 압축되며, Jasmin API에서 그 mode-2 바이트 길이는 992이다.

논문 Figure 8의 핵심 데이터 흐름을 이 문서에 필요한 수준으로만 쓰면 다음과 같다.

$$\begin{aligned} b &= a_{\text{gen}} + A_{\text{gen}} s_{\text{gen}} + e_{\text{gen}}, & (b_0, b_1) &= (\text{LowBits}_{vk}(b), \text{HighBits}_{vk}(b)), \\ vk &= (\text{seed}A, b_1), & \mu &= H_{\text{gen}}(\text{seed}A, b_1, M), \\ \rho &= H(\text{HighBits}_h(w), \text{LSB}(\lfloor y_0 \rfloor), \mu), & c &= \text{SampleBinaryChallenge}_\tau(\rho). \end{aligned}$$

Sign은 비밀키 내부의 검증키 성분을 이용해 μ 를 만들고, Verify는 공용 검증키를 이용해 이를 다시 만든다. 이어서 두 쪽은 하이비트(highbits), LSB, μ 를 도전값 전사(challenge transcript)에 넣는다. 따라서 구현 정확성의 첫 합성 지점은 단순한 “SHAKE가 올바르다”가 아니라 다음 세 개의 구체적 등식이다.

1. Sign이 읽는 패킹된 비밀키(packed secret-key) 접두부와 Verify가 읽는 패킹된 검증키(packed verification-key) 바이트가 같다.
2. 같은 pre와 메시지를 흡수한 뒤 Sign의 64바이트 μ 중 처음 32바이트가 Verify의 32바이트 μ 와 같다.
3. 그 32바이트를 같은 challenge 상태의 위치 64에서 흡수하면 최종 상태와 위치가 같다.

현재 작업은 이 세 등식을 원시/내부 경로에서 실제 생성 절차까지 연결하고, 실제 원시 API 호출자가 그 입력에 도달하는 제어/메모리 사실, Sign 출력의 프레임 조건, 읽는 구간만을 요구하는 해시 동치까지 추적했다. 완료된 추적에 저장된 두 μ 를 직접 연결하는 일은 생성/재전달 의미론 경계로 동결되었다. 현재 주 실행 경로는 실제 서명 코덱의 HBZ rANS 접미부다. 계수/기호 앞, 표, 순수 단일 단계, 공통 바이트 스택, 실제 접미부 복사와 3회 호출 결합의 제어는 단했다. Week 10에는 실제 인코더의 내부 바이트 쓰기와 성공 시 크기 범위가 추가되었다. Week 11은 순수 꼬리를 내부 불변식에 보존하는 정확한 0/1/2바이트 진행, 생성 표 단어 갱신, 외부 한 기호 전이와 마지막 네 저장을 합성하여 전체 실제 인코더 성공 접미부를 순수 추적과 동일시했다. Week 12는 같은 추적을 실제 디코더가 정확히 소비함을 담았고, Week 13은 실제 인코더-접미부 복사-디코더 결합 합성까지 담았다. 현재 관문(gate)은 이 핵심 역함수를 실제 전체 HBZ 래퍼와 prepare/apply 역함수에 합성하는 것이다.

2.2 mode-2 서명 코덱(signature codec)의 기능정확성 목표

실제 mode-2 서명은 1474바이트이며 추출된 packer/unpacker의 배치는 다음과 같다.

$$\begin{array}{ccccccc} \underline{[0, 32)} & \parallel & \underline{[32, 1056)} & \parallel & \underline{[1056, 1058)} & \parallel & \underline{[1058, 1474)} \\ \text{256 challenge bits} & & \text{1024 signed low bytes} & & \text{두 size delta} & & \text{hbz/h payload와 canonical zero padding} \end{array}.$$

마지막 영역의 총 예산(budget)은 416바이트이므로 $1056 + 2 + 416 = 1474$ 이다. 현재 달한 접두부 목표는 정준 도전값 c 와 부호 바이트 범위의 하위 배열 z_{low} 에 대해

$$\text{UnpackPrefix}(\text{PackPrefix}(c, z_{\text{low}})) = (c, z_{\text{low}})$$

이다. 이 등식은 실제 생성 절차에 대해 부분정확성으로 증명되었다. 두 rANS 적재량(payload), 크기 메타데이터(size metadata), 영 패딩(zero padding), 전체 구문 분석(parse)의 성공과 유일성은 이 등식에 포함되지 않는다. Week 8–12 결과는 그중 첫 HBZ 적재량의 내부 층을 더 세분화한다.

2.3 HBZ 앞(leaf), rANS 단일 단계와 바이트 추적(byte trace)의 수학적 구조

mode-2 HBZ 입력은 1024개의 계수 $h_i \in [-6, 7)$ 이고, 실제 prepare 절차는

$$s_i = h_i + 6 \in \{0, \dots, 12\}$$

로 옮긴다. 실제 apply 절차는 $s_i - 6$ 을 32비트 word로 복원한다. 따라서 canonical HBZ 배열에 대해 종료한 실제 두 leaf의 합성은

$$\text{Apply}(\text{Prepare}(h))[0..1024) = h[0..1024)$$

를 만족한다. prepare 정리는 별도로 용량 2048의 symbol tail을 보존하고, 이 합성 정리의 사후조건은 복호 계수의 꼬리를 보존한다. 이 결과는 rANS 버퍼(buffer)의 성공이나 크기를 말하지 않는 앞 수준 부분정확성이다.

13개 symbol의 누적 시작점과 빈도는 각각

$$(c_s)_{s=0}^{12} = (0, 1, 2, 3, 8, 66, 312, 710, 957, 1016, 1021, 1022, 1023),$$

$$(f_s)_{s=0}^{12} = (1, 1, 1, 5, 58, 246, 398, 247, 59, 5, 1, 1, 1),$$

이며 $f_s > 0$, $\sum_s f_s = 1024$ 이고 구간 $[c_s, c_s + f_s)$ 들이 $[0, 1024)$ 를 분할한다. 순수 rANS 단계는

$$E_s(x) = \left\lfloor \frac{x}{f_s} \right\rfloor 1024 + c_s + (x \bmod f_s),$$

$$D_s(y) = \left\lfloor \frac{y}{1024} \right\rfloor f_s + (y \bmod 1024) - c_s$$

로 두며, 현재 증명은 $x \geq 0$ 에서 $D_s(E_s(x)) = x$ 를 보인다. 더 나아가 실제 mode-2 역수/편향(reciprocal/bias) 식으로 계산하는 빠른 단계(fast step)가 $0 \leq s < 13$, $1 \leq x < \text{hbz_xmax}(s)$ 에서 $E_s(x)$ 와 같음도 증명되었다. 실제 패킹/언패킹 추출물의 리터럴(literal) esyms/dsyms_words/symbol_words 배열이 이 certificate를 만족한다.

Week 9은 단일 단계 사이를 잇는 순수 바이트 추적을 다음처럼 고정했다. 기호열을 $s_0, \dots, s_{n-1}$, $n = 1024$ 라 하고 $x_n = L = 2^{23}$ 로 둔다. encoder 방향 $j = n - 1, \dots, 0$ 에서

$$\ell_j = \text{renorm_len}(x_{j+1}, \text{hbz_xmax}(s_j)) \in \{0, 1, 2\},$$

$$r_j = \left\lfloor x_{j+1} / 256^{\ell_j} \right\rfloor,$$

$$x_j = E_{s_j}(r_j).$$

이때 β_j 를 실제 디코더가 읽는 순서의 정규화 바이트 구간(segment)이라 하면, 컴파일된 readback 정리는

$$\text{ReadBytes}(r_j, \beta_j) = x_{j+1}$$

를 준다. 특히 두 바이트를 내보낼 때 실제 인코더는 하위 바이트를 먼저 쓰면서 커서(cursor)를 감소시키므로 최종 순방향 구간은 $[\text{high}, \text{low}]$ 다. 따라서

$$c_0 = 4, \quad c_{j+1} = c_j + |\beta_j|, \quad B = \text{LE32}(x_0) \parallel \beta_0 \parallel \dots \parallel \beta_{n-1}$$

를 공통 추적으로 사용한다. ParseLE32가 첫 상태를 복원하고, 각 D_{s_j} 가 x_j 에서 r_j 를 얻은 뒤 β_j 를 읽어 x_{j+1} 를 복원한다. 순수 귀납 정리는 모든 boundary state에 대해

$$2^{23} \leq x_j < 2^{31}$$

을 보존한다.

이 순수 결과만으로 실제 인코더와 디코더의 외부 반복문 정제가 따라오지는 않는다. 실제 __copy_encoded_suffix가 인코더 접미부를 디코더용 접두부로 점별 복사하고 남은 꼬리를 보존한다는 Hoare 정리와, 실제 인코더의 반환 bad에 따라 복사와 디코더 호출을 수행하거나 생략하는 단항 결합의 제어 정리까지는 컴파일된다.

2.4 Week 10 실제 인코더의 수학적 경계

용량 2048의 기호 배열 a 에서 mode-2가 사용하는 목록과 그 접미부를

$$S(a) = [\text{uint}(a[0]), \dots, \text{uint}(a[1023])], \quad S_i(a) = \text{drop}_i(S(a))$$

로 둔다. 코드의 `symbol_list_of_array`와 `symbol_suffix`가 바로 이 정의다. 배열 구간과 프레임 조건은

$$\begin{aligned} \text{Segment}(A, p, b) &\iff 0 \leq p \wedge p + |b| \leq 2048 \wedge \\ &\quad \forall k \in [0, |b|), \text{uint}(A[p + k]) = b[k], \\ \text{PrefixFrame}(A_0, A_1, p) &\iff \forall k \in [0, p), A_0[k] = A_1[k] \end{aligned}$$

로 읽는다. 컴파일된 배열/목록 연결 정리는 $S_{1024}(a) = []$ 와 $S_i(a) = \text{uint}(a[i]) :: S_{i+1}(a)$ 를 주며, 한 기호 추적의 점화식도 다음처럼 분리한다.

$$\begin{aligned} T(s :: t).\text{state} &= E_s(\text{Norm}(T(t).\text{state}, s)), \\ T(s :: t).\text{bytes} &= \text{NormBytes}(T(t).\text{state}, s) \| T(t).\text{bytes}. \end{aligned}$$

실제 표의 네 단어, 64비트 역수 곱과 상위 절반 추출, 이동 사다리와 편향을 모델링한 단어 단계는

$$\begin{aligned} 0 \leq s < 13, \quad 1 \leq \text{uint}(x) < \text{hbz_xmax}(s), \\ \text{ActualWordStep}(x, s) &= \text{W32}(\text{FastStep}(\text{uint}(x), s)) \end{aligned}$$

를 만족한다. 앞선 빠른 단계 정리와 합치면 오른쪽은 수학적 E_s 의 W32 표현이다. 이것은 단어 수준 등식이지 생성된 외부 반복문에 대한 Hoare 정리는 아니다.

생성된 `_rans_encode`의 내부 정규화 반복문에는 진입 상태 (x_0, off_0) , 임계값 $x_{\max}$ 와 단계 k 를 증인으로 쓰는 다음 국소 불변식이 설치되었다.

$$\begin{aligned} 4 \leq \text{off}_0 \leq 1024, \quad 0 \leq k \leq 4, \quad 1 \leq \text{uint}(x_{\max}), \\ \text{off} = \text{off}_0 - k, \quad x = \text{PhaseState}(x_0, k), \\ \text{Segment}(\text{encp}, \text{off}_0 - k, \text{PhaseBytes}(x_0, k)), \quad \text{PrefixFrame}(\text{before}, \text{encp}, \text{off}_0 - k). \end{aligned}$$

각 실제 `set8` 단계가 낮은 바이트를 목록 앞에 붙이고, W64 커서가 언더플로하지 않으며, 쓰지 않은 접두부를 보존한다는 사실이 직접 증명된다. 다만 `before`는 해당 내부 반복문 진입 시 배열이므로 이 불변식은 누적된 전체 외부 추적을 아직 나타내지 않는다. $0 \leq k \leq 4$ 도 W32 유한성에서 얻은 보수적 범위이며, 순수 추적의 정확한 0/1/2바이트 길이와의 동일시는 남아 있다.

그 결과 실제 생성 절차에 대한 두 부분정확성 정리는

$$\begin{aligned} \text{bad}' \neq 0 \vee (\text{bad}' = 0 \wedge 0 \leq \text{off}' \leq 1020), \\ \text{bad}' = 0 \implies 4 \leq 1024 - \text{off}' \leq 1024 \end{aligned}$$

를 준다. 두 번째 식은 성공 시 인코딩 크기 범위다. 성공 경로의 도달 가능성이나 종료, 모든 기호가 6인 입력(all-6 input)의 성공은 결론에 포함되지 않는다.

2.5 Week 11 실제 인코더 추적 폐쇄

Week 11의 꼬리 인식 내부 불변식은 국소 바이트만 기억하지 않고 이미 작성된 순수 꼬리(pure tail) t 를 함께 운반한다.

$$\begin{aligned} 0 \leq k \leq \ell(x_0, s), \quad \text{off} = \text{off}_0 - k, \\ \text{Segment}(\text{encp}, \text{off}_0 - k, \text{Written}(x_0, s, k) \| T(t).\text{bytes}), \quad \text{PrefixFrame}(\text{enc}_0, \text{encp}, \text{off}_0 - k). \end{aligned}$$

여기서 $\ell(x_0, s) \in \{0, 1, 2\}$ 는 순수 정규화 길이다. 실제 생성 가드(guard)가 거짓이 되는 지점에서 $k = \ell(x_0, s)$ 임이 증명되므로 Week 10의 보수적 0.4 위상과 순수 추적 사이의 간극이 닫힌다.

성공 경로의 외부 불변식은

$$\begin{aligned} 0 \leq i \leq 1024, \quad x &= T(S_i(a_0)).\text{state}, \quad 4 \leq \text{off}, \\ \text{off} + |T(S_i(a_0)).\text{bytes}| &= 1024, \\ \text{Segment}(\text{encp}, \text{off}, T(S_i(a_0)).\text{bytes}), \quad &\text{PrefixFrame}(\text{enc}_0, \text{encp}, \text{off}) \end{aligned}$$

이다. 생성 표 읽기와 보호 연산, 역수 곱과 이동 사다리를 전용 단어 정리로 접어 한 기호 전이를 복원하고, $i = 0$ 에서 네 번의 실제 리틀엔디언 저장을 합성한다. 그 결과 실제 `_rans_encode`의 종료한 실행은 실패를 반환하거나, 성공 시 다음 사후조건을 만족한다.

$$\begin{aligned} \text{bad}' \neq 0 \quad \vee \quad (\text{bad}' = 0 \wedge 0 \leq \text{off}' \leq 1020 \wedge 4 \leq 1024 - \text{off}' \leq 1024 \\ \wedge \text{Segment}(\text{encp}', \text{off}', \text{trace_bytes}(S(a_0))) \\ \wedge \text{off}' + |\text{trace_bytes}(S(a_0))| = 1024 \\ \wedge \text{PrefixFrame}(\text{enc}_0, \text{encp}', \text{off}')). \end{aligned}$$

`actual_rans_encode_trace_closure`와 `actual_rans_encode_trace_refinement`가 이 명제를 실제 생성 절차에 대해 증명한다. 따라서 **OBL-RANS-ENCODE-REFINEMENT**는 성공 조건부 부분정확성으로 **PROVED**다. 성공은 사전조건이 아니지만, 정리가 종료나 $\text{bad}' = 0$ 의 도달 가능성을 보이는 것도 아니다.

2.6 Week 12 실제 디코더 추적 폐쇄

용량 2048의 예상 기호 배열 a 에 대해

$$s := S(a), \quad B := \text{trace_bytes}(s), \quad n := |B|$$

로 둔다. 실제 디코더의 정확 추적 입력(exact-trace input)은 `mode2_hbz_symbol_stream(a)`, $4 \leq n \leq 1024$, 입력 버퍼의 $[0, n)$ 구간과 B 의 점별 일치, 상태 필드

$$(\text{count}, \text{size}, m) = (1024, n, 13)$$

및 실제 `symbol_words/dsyms_words` 표의 바인딩을 요구한다. 각 정규화 구간 안에서 생성된 바이트 읽기가 해당 순수 목록 원소와 같다는 점별 읽기 전제도 명시한다. 이 입력 술어에는 디코더 성공, 출력 기호 등식이나 사후상태가 들어 있지 않다.

참조 상태, 커서와 구간을

$$\begin{aligned} x_i &:= \text{decoder_state_at}(s, i), \quad c_i := \text{decoder_cursor}(s, i), \\ \beta_i &:= \text{decoder_segment_at}(s, i) \end{aligned}$$

로 두면 컴파일된 경계 정리는

$$c_0 = 4, \quad c_{i+1} = c_i + |\beta_i|, \quad c_{1024} = n, \quad x_{1024} = 2^{23}$$

을 준다. 실제 내부 반복문은 $k = \text{off} - c_i$ 와 부분 재생 상태를 운반하며, 구간 길이 $|\beta_i| \in \{0, 1, 2\}$ 에 따라 정확히 다음 바이트를 소비하거나 멈춘다. 구체적인 패킹 표 조회와 W32/W64 단어 연산은 같은 기호와 수학적 디코더 단계를 선택함이 별도 정리로 연결된다.

그 결과 `actual_rans_decode_trace_refinement`는 실제 생성 `RansDecodeTarget.M._rans_decode`의 모든 종료 실행에 대해

$$\begin{aligned} \text{bad}_{\text{dec}} &= 0, \quad \text{off}_{\text{dec}} = n, \\ \text{decoded}'[0..1024) &= a[0..1024), \quad \text{decoded}'[1024..2048) = \text{decoded}_0[1024..2048) \end{aligned}$$

을 증명한다. 내부 사실 $x_{1024} = 2^{23}$ 은 실제 마지막 거절 검사를 배제하는 데 쓰이지만 생성 ABI 결과에는 노출되지 않는다. 따라서 **OBL-RANS-DECODE-REFINEMENT**는 정확 추적 부분정확성으로 **PROVED**다. 종료, 손실없음(losslessness), 실제 인코더 성공 도달은 이 정리의 결론이 아니다.

2.7 Week 13 실제 rANS 핵심 합성 폐쇄

Week 13은 앞 절의 세 개별 정리를 실제 `Mode2RansActualHarness.run` 안에서 순차 합성했다. 이 결합 모듈은 고정 추출물의 `_rans_encode`, `__copy_encoded_suffix`, `_rans_decode`를 그 순서로 직접 호출한다. `Mode2RansCoreCompositionBridge.ec`는 인코더의 `segment_matches`와 실제 복사의 `slice_eq`에서 복사된 버퍼의 정확 추적을 만들고, 이를 모든 디코더 점별 읽기와 구성된 상태 필드 $(1024, n, 13)$ 로 운반한다. 또한 실제 성공 범위로부터 W64 뺄셈이 정수 추적 크기와 같고 랩어라운드하지 않음을 도출한다.

`actual_rans_encode_copy_decode_inverse`의 사전조건은 초기 인자 바인딩과 `mode2_hbz_symbol_stream`뿐이다. 특히 인코더 성공이나 복사/디코더 결과를 사전조건으로 넣지 않는다. 모든 종료 실행의 결론은 대략

$$\begin{aligned} & (bad_{enc} \neq 0 \wedge \neg decoder_ran \wedge decoded' = decoded_0) \\ \vee & (bad_{enc} = 0 \wedge decoder_ran \wedge bad_{dec} = 0 \wedge off_{dec} = size \\ & \wedge decoded[0..1024) = symbols[0..1024) \wedge frames) \end{aligned}$$

이다. 실패 분기에서는 디코더 상태도 초기값으로 남고, 성공 분기에서는 디코더 입력 필드, 실제 인코더 추적 접미부와 초기 접두부 프레임까지 보존된다. 따라서 **OBL-RANS-CORE-INVERSE**는 성공 조건부 Hoare 부분정확성으로 **PROVED**다.

이 결과는 세 실제 생성 절차를 직접 호출하는 증명 작성 결합 모듈의 정리이지, `production_encode_hb_z1_full/_decode_hb_z1_full` 래퍼 정리는 아니다. 정준 h 만으로 버퍼 성공을 결론낼 수도 없으므로 전체 HBZ 래퍼(wrapper) 목표는 여전히 적어도

$$size = 0 \quad \vee \quad (size \neq 0 \wedge bad = 0 \wedge decoded[0..1024) = h[0..1024))$$

처럼 실제 실패 분기를 드러내야 한다. HBZ 부모의 전체 프레임/크기 운반, 인코더와 디코더의 종료, 그리고 모든 기호가 6인 입력의 실제 성공 증인은 아직 컴파일되지 않았다.

2.8 최상위 기능정확성 목표

`easycrypt/specs/TopLevelGoals.ec`는 구현과 논문의 확률적 알고리즘을 추상 연산으로 두고 다음 목표를 정의한다.

$$\begin{aligned} FC_{KG} &: JKeyGen_2 = PaperKeyGen_2, \\ FC_{Sign} &: \forall sk, M, JSign_2(sk, M) = PaperSign_2(sk, M), \\ FC_{Verify} &: \forall vk, M, \sigma, JVerify_2(vk, M, \sigma) = PaperVerify_2(vk, M, \sigma). \end{aligned}$$

이 식들은 증명 완료 선언이 아니라 앞으로 구현해야 할 명세 인터페이스이다. 현재의 `prefix`와 `transcript` 정리는 이 등식들에 필요한 결정론적 하위 정리다.

2.9 최상위 보안 합성 목표

구현의 EUF-CMA 이점을 논문 스킴의 이점과 비교하는 목표는

$$Adv_{Jasmin}^{euf-cma} \leq Adv_{paper}^{euf-cma} + \delta_{KeyGen} + q_{Sign} \delta_{Sign} + \delta_{Verify} + \delta_{Encoding} \quad (1)$$

이다. 논문 수준에서는 다시

$$Adv_{paper}^{euf-cma} \leq Adv_{MLWE} + Adv_{BST-MSIS} + \epsilon_{ROM} + \epsilon_{Rejection} + \epsilon_{Fork} \quad (2)$$

을 목표로 한다. [equations \(1\) and \(2\)](#) 역시 현재는 **SPECIFIED** 또는 조건부 연결식이다. 최종적으로 남길 수 있는 암호학적 가정 인터페이스는 HAETAE-2의 MLWE, BST-MSIS, ROM 세 종류지만, 구체적 게임 인스턴스와 감소정리는 아직 모두 연결되지 않았다.

3 명제를 읽기 위한 형식 언어(EasyCrypt)

3.1 핵심 술어와 관계

정의 3.1 (패킹된 키(packed-key) 접두부).

$$\text{VKPrefix}_n(sk, vk) \iff \forall i (0 \leq i < n \Rightarrow sk[i] = vk[i]).$$

코드의 `vk_prefix_eq`와 같으며, mode 2에서는 $n = 992$ 이다.

정의 3.2 (외부 키(key) 영역의 접두부 일치).

$$\text{ExternalKeyPrefix}(mem, sku, vku) \iff \forall i \in [0, 992), \text{load8}(mem, sku+i) = \text{load8}(mem, vku+i).$$

코드의 `external_key_prefix_match`이며, 실제 raw KeyGen이 서로 다른 외부 SK/VK 주소에 내보낸 첫 992바이트를 비교한다.

정의 3.3 (메모리에서의 접두부).

$$\text{SKMemPrefix}(sk, mem, b) \iff \forall i \in [0, 992), sk[i] = \text{load8}(mem, \text{uint}(b) + i).$$

이 술어는 Sign의 로컬 BArray2752와 Verify의 raw-memory 읽기를 한 관계식 안에서 비교한다.

정의 3.4 (정준 주소 구간(canonical address range)). 정수 주소 a 와 길이 n 에 대해

$$\begin{aligned} \text{CanonicalAddress}(a) &\iff 0 \leq a < 2^{64}, \\ \text{CanonicalRegion}(a, n) &\iff 0 \leq a \wedge 0 \leq n \wedge a + n \leq 2^{64}. \end{aligned}$$

첫 전제는 원시 응용 이진 인터페이스(raw application binary interface, raw ABI)의 정수 주소가 `W64.of_int`와 `W64.to_uint`를 왕복하게 하고, 둘째는 주소 구간의 모듈러 순환(wraparound)을 배제한다. 길이가 0이면 $a = 2^{64}$ 도 둘째 식만으로는 허용되므로, 실제 왕복 정리는 필요한 주소와 길이에 `canonical_ui64_address`를 별도로 요구한다. 이것은 물리적 메모리 안전성 증명이 아니라 호출 계약이다.

정의 3.5 (메모리 구간 등식).

$$\text{RegionEq}(m_1, m_2, b, n) \iff \forall i \in [0, \text{uint}(n)), \text{load8}(m_1, \text{uint}(b) + i) = \text{load8}(m_2, \text{uint}(b) + i).$$

코드의 `region_eq`다. Week 6의 hash 정리는 전체 memory 등식 대신 VK, pre, message에서 실제로 읽는 구간의 이 관계만 요구한다.

정의 3.6 (정준 서명 접두부(canonical signature prefix) 입력). `canonical_challenge`는 256개 challenge word가 각각 정확히 0 또는 1임을 뜻한다. `canonical_signed_low`는 1024개 low word 각각이 자신의 하위 바이트를 부호확장한 값이고 signed 정수로 $[-128, 128)$ 에 있음을 뜻한다. 실제 prefix round trip은 이 두 전제 아래 성립한다.

정의 3.7 (정준 mode-2 HBZ와 잎 프레임(leaf frame)). 용량 2048의 32비트 계수 배열 h 에 대해

$$\text{CanonicalHBZ}_2(h) \iff \forall i \in [0, 1024), -6 \leq \text{sint}(h[i]) < 7.$$

코드의 `canonical_hbz_mode2`다. `prepared_hbz_prefix`는 첫 n 개 symbol byte가 $\text{sint}(h[i]) + 6$ 임을, `decoded_hbz_prefix`는 첫 n 개 복원 계수가 원본과 같음을 뜻한다. `byte_tail_frame`과 `coeff_tail_frame`은 각각 $[1024, 2048)$ 이 leaf 실행 전후에 변하지 않음을 기록한다.

정의 3.8 (mode-2 HBZ 표 인증서(table certificate)). `mode2_hbz_table_certificate`는 다음 세 사실의 conjunction이다.

1. 13개 symbol의 실제 encoder table 네 field가 명세식과 같다.
2. 실제 decoder word가 각 interval의 start/frequency를 정확히 담는다.
3. 실제 512개의 packed symbol word가 1024개 slot 각각을 유일한 interval symbol에 연결한다.

actual_mode2_hbz_tables_certified는 pack/unpack 추출물의 literal 배열에 이 술어를 적용한 정리다. certificate를 생성한 외부 스크립트 자체를 가정하는 것이 아니라, 생성된 모든 EasyCrypt 등식을 kernel이 다시 검사한다.

정의 3.9 (정준 rANS 바이트 추적(canonical rANS byte trace)). 정준 기호 목록(canonical symbol list) $s = (s_0, \dots, s_{1023})$ 에 대해 encode_trace는 $x_{1024} = 2^{23}$ 에서 기호(symbol)를 역순으로 처리해 경계 상태(boundary state) $(x_j)_{j=0}^{1024}$ 와 디코더 순서 바이트 구간(decoder-order byte segment) $(\beta_j)_{j=0}^{1023}$ 를 결정한다. 커서 절단값(cursor cut)과 전체 바이트열(byte sequence)은

$$c_0 = 4, \quad c_{j+1} = c_j + |\beta_j|, \quad B = \text{serialize32_le}(x_0) \parallel \beta_0 \parallel \dots \parallel \beta_{1023}$$

이다. 코드의 valid_rans_trace(s,xs,cuts,bytes)는 단지 trace_states, trace_cuts, trace_bytes로 계산한 이 세 객체와 인자가 같다는 논리곱(conjunction)이다. 외부 추적(external trace)의 존재나 디코더 성공(decoder success)을 가정하는 술어가 아니며, valid_rans_trace_canonical이 정준 구성(canonical construction)을 증명한다.

정의 3.10 (배열/목록 연결과 구간 프레임(array/list bridge and segment frame)). 용량 2048의 바이트 배열 a 에 대해 symbol_list_of_array(a)는 앞 1024개 원소의 정수 목록이고, symbol_suffix(a,i)는 그 목록에서 앞 i 개를 버린 접미부다. segment_matches(a,p,bs)는

$$0 \leq p \wedge p + |bs| \leq 2048 \wedge \forall k \in [0, |bs|), \quad \text{uint}(a[p+k]) = bs[k]$$

를 뜻한다. prefix_frame(before,after,p)는 모든 $k < p$ 에서 두 배열의 k 번째 바이트가 같다는 뜻이다. Week 10의 컴파일된 정리는

$$\begin{aligned} \text{symbol_suffix}(a, 1024) &= [], \\ \text{symbol_suffix}(a, i) &= \text{uint}(a[i]) :: \text{symbol_suffix}(a, i + 1) \end{aligned}$$

와 구간의 이어 붙이기, 앞쪽 한 바이트 쓰기, 프레임 축소 규칙을 제공한다.

정의 3.11 (실제 인코더 내부 단계 불변식(actual encoder inner-step invariant)). encoder_inner_phase_inv와 encoder_inner_segment_inv는 생성된 _rans_encode의 내부 정규화 반복문을 직접 연다. 진입 상태 (x_0, off_0) 와 단계 k 가 존재하여

$$\begin{aligned} 4 \leq off_0 \leq 1024, \quad 0 \leq k \leq 4, \quad 1 \leq \text{uint}(x_{\max}), \\ \text{uint}(off) = off_0 - k, \quad \text{uint}(x) = \text{encoder_phase_state}(x_0, k), \\ \text{segment_matches}(encp, off_0 - k, \text{encoder_phase_written}(x_0, k)) \end{aligned}$$

이고 해당 내부 호출의 진입 배열보다 낮은 접두부가 보존된다는 뜻이다. 이는 Week 10의 보수적 국소 술어로서 실제 한 바이트 저장과 W64 커서 안전성을 검증한다. Week 11의 더 강한 꼬리 인식 술어가 외부 누적과 정확한 0/1/2바이트 길이를 추가한다.

정의 3.12 (꼬리 인식 내부 불변식(tail-aware inner invariant)). Week 11의 encoder_inner_tail_inv는 정확한 순수 정규화 길이 $\ell = \text{mode2_normalization_len}(x_0, s)$ 와 이미 존재하는 순수 기호 꼬리 tail의 인코딩 추적을 함께 둔다. 핵심 구간 조건은

$$\text{segment_matches}(encp, off_0 - k, \text{inner_written_suffix}(x_0, s, k) ++ \text{encode_trace}(tail).bytes)$$

이고 $0 \leq k \leq \ell$, 커서 등식, 상태 등식과 접두부 프레임을 함께 요구한다. 실제 가드(actual guard)의 진행과 종료에서 이 술어가 보존되고 정확한 $k = \ell \leq 2$ 가 도출된다. 이어 외부 한 기호 점화식과 최종 저장 합성에 사용되어 실제 외부 절차의 전체 성공 사후조건을 달는다.

검증된 정리 3.1 (실제 인코더의 성공 조건부 전체 접미부). `actual_rans_encode_trace_closure`는 실제 생성 `_rans_encode`를 직접 열고, 반환 $bad \neq 0$ 이거나 $bad = 0$ 일 때

$$\text{segment_matches}(\text{encp}', \text{uint}(\text{off}'), \text{trace_bytes}(\text{symbol_list_of_array}(\text{symbols}_0)))$$

와 $\text{off}' + |\text{trace_bytes}| = 1024$, 성공 크기 범위 및 `prefix_frame`이 모두 성립함을 증명한다. 성공은 사전조건이 아니다. 그러나 Hoare 부분정확성이므로 종료와 성공 도달은 별도 의무다.

정의 3.13 (실제 디코더 정확 추적 입력(actual decoder exact-trace input)). `actual_mode2_decoder_trace_input(a,b,st,n)`은 예상 기호 배열 a , 인코딩 버퍼 b , 초기 상태 st 와 논리적 크기 n 사이의 함수적 표현 계약이다. 핵심은

$$\begin{aligned} \text{mode2_hzb_symbol_stream}(a), \quad n = |\text{trace_bytes}(\text{symbol_list_of_array}(a))|, \\ 4 \leq n \leq 1024, \quad \text{segment_matches}(b, 0, \text{trace_bytes}(\dots)), \\ (st[0], st[1], st[2]) = (1024, n, 13) \end{aligned}$$

와 모든 정규화 구간에 대한 점별 읽기 등식이다. 실제 `mode-2 symbol_words`와 `dsyms_words` 배열은 최상위 Hoare 정리의 별도 바인딩으로 고정된다. 이 술어는 $bad = 0$, 출력 배열 등식 또는 사후상태를 전제로 넣지 않는다.

정의 3.14 (디코더 외부/내부 추적 불변식(decoder outer/inner trace invariant)). `decoder_outer_trace_live`는 반복 번호 i 에서 실제 상태와 커서를

$$x = \text{decoder_state_at}(s, i), \quad \text{uint}(\text{off}) = \text{decoder_cursor}(s, i)$$

에 묶고, `decoded[0..i)`의 예상 기호 일치와 `decoded[i..2048)`의 입력 배열 프레임을 함께 유지한다. `decoder_inner_trace_live`는 한 기호의 단어 갱신 뒤

$$\begin{aligned} k = \text{uint}(\text{off}) - \text{decoder_cursor}(s, i), \\ x = \text{decoder_replay_prefix}(\text{decoder_state_at}(s, i + 1), s_i, k) \end{aligned}$$

를 두고 $0 \leq k \leq |\text{decoder_segment_at}(s, i)|$ 를 유지한다. 생성 가드는 정확히 $k < |\text{segment}|$ 와 일치하며 구간 길이는 0, 1 또는 2바이트다. 따라서 각 실제 읽기는 한 바이트 소비 전이이거나 다음 외부 경계로 가는 무소비(no-consume) 종료다.

검증된 정리 3.2 (실제 디코더의 정확 추적 복원). `actual_rans_decode_trace_refinement`는 **정의 3.13**과 실제 두 표의 바인딩 아래 실제 생성 `RansDecodeTarget.M._rans_decode`를 직접 연다. 모든 종료 실행에서

$$bad' = 0, \quad \text{off}' = n, \quad \text{decoded}'[0..1024) = a[0..1024)$$

이고 `[1024, 2048)` 출력 꼬리가 초기 배열과 같다. 외부 반복 출구의 내부 $x = 2^{23}$ 는 마지막 실제 거절 검사를 배제하지만 ABI 사후조건에는 나타나지 않는다. 이는 정확 추적 Hoare 부분정확성이며 종료나 실제 인코더 성공 증명은 아니다.

정의 3.15 (실제 rANS 결합 제어(actual rANS harness control)). `Mode2RansActualHarness.run`은 실제 `_rans_encode`를 호출한 뒤 반환된 bad 가 0일 때만 실제 `_copy_encoded_suffix`와 `_rans_decode`를 호출한다. 디코더 상태의 입력 필드는 그 성공 분기에서 정확히 $(\text{count}, \text{size}, m) = (1024, \text{encoded_size}, 13)$ 이다. 현재 결합 제어 정리는

$$bad_{\text{enc}} \in \{0, 1\}, \quad \text{decoder_ran} \iff bad_{\text{enc}} = 0$$

과 이 필드 초기화만 결론낸다. 이 제어 정리 자체는 역함수가 아니다. Week 13의 별도 의미 정리는 **검증된 정리 3.1**와 **검증된 정리 3.2**, 실제 복사 정리를 이 모듈 안에서 순차 호출해 다음 결론을 달는다.

검증된 정리 3.3 (실제 인코더-복사-디코더 핵심 역함수). `actual_rans_encode_copy_decode_inverse`는 초기 인자 바인딩과 `mode2_hbz_symbol_stream`만을 전제로 한다. 종료한 실행에서 실제 인코더가 실패하면 디코더는 실행되지 않고 초기 디코더 배열/상태가 보존된다. 실제 인코더가 성공하면 복사된 버퍼가 정확한 `trace_bytes` 입력이 되고, 디코더는

$$bad_{dec} = 0, \quad off_{dec} = encoded_size, \quad decoded[0..1024) = symbols[0..1024)$$

를 만족하며 `[1024, 2048)` 꼬리를 보존한다. 인코더 성공, 복사 구간 등식, 디코더 성공이나 출력 등식은 사전조건이 아니다. 이 정리는 Hoare 부분정확성이므로 성공 도달과 두 반복문의 종료는 별도 의무다.

경계 명제 3.1 (성공 조건부(success-conditioned) HBZ). `CanonicalHBZ2(h)`는 prepare leaf가 성공함을 보장하지만 고정 1024바이트 rANS workspace의 actual full encoder가 nonzero size를 반환함을 보장하지 않는다. 따라서 full round trip에서 encoder success나 decoder success를 독립된 사전조건으로 가정하면 비공허성이 사라진다. 실제 return의 `size = 0` branch를 결론에 남기거나, actual successful witness를 별도로 증명해야 한다.

정의 3.16 (μ 접두부). Sign이 만드는 64바이트 배열을 μ_S^{64} , Verify가 만드는 32바이트 배열을 μ_V^{32} 라 하면

$$\text{MuPrefix}_{32}(\mu_S^{64}, \mu_V^{32}) \iff \forall i \in [0, 32), \mu_S^{64}[i] = \mu_V^{32}[i].$$

즉 $\text{take}_{32}(\mu_S^{64}) = \mu_V^{32}$ 이다.

정의 3.17 (원시 분기(raw branch)와 유효 구간). 64비트 word p, ℓ 에 대해 코드의 두 술어는

$$\text{RawPreLen}(\ell) \iff 0 \leq \text{uint}(\ell) < 2^{63}, \quad \text{ValidRegion}(p, \ell) \iff \text{uint}(p) + \text{uint}(\ell) \leq 2^{64}$$

이다. 첫 식은 Verify가 `prelen >> 63`으로 고르는 실제 분기에서 raw branch를 선택하게 하고, 둘째 식은 주소 덧셈의 wraparound를 배제한다.

3.2 Hoare 명제와 관계적 동치

EasyCrypt의

$$\text{hoare}[P : \Phi \implies \Psi]$$

는 P 가 종료한다는 명제가 아니다. Φ 에서 시작하여 종료한 모든 실행이 Ψ 를 만족한다는 부분정확성 명제다. 따라서 KeyGen의 retry loop가 lossless인지, 거의 확실히 종료하는지, 논문과 같은 분포를 내는지는 별도 정리가 필요하다.

관계적 명제

$$\text{equiv}[P \sim Q : \Phi \implies \Psi]$$

는 두 실행 상태의 사전관계 Φ 에서 시작한 P, Q 의 결과가 사후관계 Ψ 를 만족함을 뜻한다. 본 작업의 hash 정리는 SHAKE를 추상 함수로 치환하지 않고, 양쪽의 생성된 absorb/finalize/Keccak/squeeze 절차를 이 관계 논리로 직접 비교한다.

3.3 정확 추적(exact trace)과 승인 꼬리(accepted tail)

새로운 raw API 정리의 trace module은 실제 절차의 제어흐름을 복사하면서 `observed_mu`, `pointer`, `length`, `tail_reached` 같은 ghost 관찰값만 추가한다. exact equivalence는 실제 절차와 trace가 최종 `Glob.mem`과 반환값을 같게 만든다는 뜻이지, 모든 실행이 hash tail에 도달한다는 뜻은 아니다.

Verify는 서명 길이, unpack, norm 등의 검사를 통과하기 전에 거절할 수 있다. 현재 정리는 이 분기를 숨기지 않고

$$\text{VerifyAccept} \implies \text{TailReached}$$

를 증명한 뒤, 성공한 실행에서만 실제 `__verify_hash_mu`에 전달된 VK/pre/message 주소와 길이를 관찰값에 연결한다. 이는 “임의의 서명이 tail에 도달한다”는 주장과 구별해야 한다.

exact trace가 실제 절차와 같은 결과를 보존한다고 해서 trace 내부 hash 호출의 반환값과 종료 후 ghost 필드가 자동으로 관계 정리에 들어오는 것은 아니다. 현재 남은 **OBL-MU-PRODUCT-REPLAY**는 두 completed sequential trace의 observed_mu를 기존 pRHL hash-return 정리에 연결하는 규칙이다. EasyCrypt의 단순한 sim, seq, byequiv만으로 서로 다른 residual continuation을 제거할 수 없어 이 간선은 **DEFERRED**로 명시되었다.

3.4 배열 크기와 논리적 길이

추출된 이론은 여러 파라미터 집합을 담을 수 있는 고정 배열 BArray2080, BArray2752, BArray2948을 사용한다. 이것은 mode-2의 논리적 길이가 각각 2080, 2752, 2948이라는 뜻이 아니다. 실제 mode-2 API 길이는 검증키 992, 비밀키 1408, 서명 1474바이트이고, 현재 정리는 그중 첫 992바이트의 관계를 추적한다. 서명 codec 쪽에서는 별도로 challenge 32바이트와 low 1024바이트, 즉 서명의 첫 1056바이트 배치와 round trip을 추적한다. HBZ leaf 쪽의 BArray8192는 2048개의 32비트 계수를, symbol BArray2048은 2048바이트를 담지만 mode-2 논리적 count는 1024다. 따라서 Week 8 frame 정리는 두 배열의 [1024, 2048) tail 보존을 별도로 명시한다.

4 현재 닫힌 증명 사슬

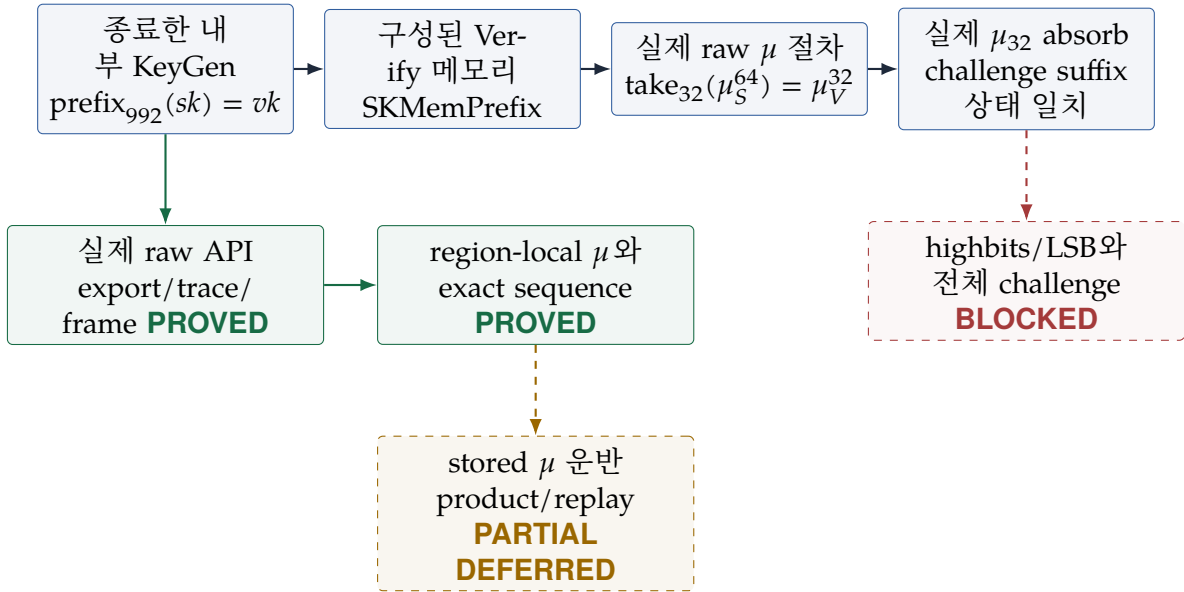


그림 1: 검증된 raw/internal 사슬(위)과 Week 6/7 API 의미론 경계(아래). 이 보안 경계는 Week 8-9에도 그대로 유지된다.

4.1 1단계: KeyGen이 반환하는 패킹된 키(packed-key) 접두부

Jasmin `pack_sk_m23`는 먼저 `vkp[0..vkbytes)`를 `skp`에 복사한다. mode 2에서 `vkbytes=992`이고, 이후 쓰기는 다음 배치를 따른다.

$$\underbrace{[0, 992)}_{vk} \parallel \underbrace{[992, 1184)}_{3.64 \text{ 바이트 } \eta_1 \text{ 벡터}} \parallel \underbrace{[1184, 1376)}_{2.96 \text{ 바이트 } \eta_2 \text{ 벡터}} \parallel \underbrace{[1376, 1408)}_{32 \text{ 바이트 키}}$$

`pack_vec_eta_to_prefix_frame`와 `pack_vec2_eta_to_prefix_frame`는 뒤쪽 쓰기가 `[0, 992)`를 바꾸지 않음을 증명한다.

검증된 정리 4.1 (생성된 패커(packer)의 접두부). `pack_sk_m23_mode2_vk_prefix`는 mode-2 상수 아래 실제 생성된 `_pack_sk_m23`가 종료하면

$$\text{VKPrefix}_{992}(\text{res}, vk_0)$$

임을 증명한다. 이 결과는 `keypair_full_m23_mode2_return_prefix`와 `keypair_internal_mode2_return_prefix`를 통해 실제 생성된 `_keypair_full_m23` 및 `crypto_sign_keypair_internal_mode2_jazz`의 반환값까지 올라간다.

이 정리는 KeyGen의 모든 retry 시도 내부를 분포적으로 분석하지 않아도 성립한다. 어떤 시도에서 종료하든 마지막 packer 호출이 같은 접두부를 만들기 때문이다. 정확히 그 이유로 정리는 강한 반환값 불변식이지만 여전히 부분정확성이다.

4.2 2단계: 로컬 배열에서 Verify 메모리로

`Mode2KeyMemoryBridge.stored_vk_buffer_at`는 기존 메모리 위에 검증키의 첫 992바이트를 stores로 기록한 구체적 메모리를 만든다. `keygen_prefix_memory_relation_is_constructible`는 정의 3.1를 사용해 그 메모리가 정의 3.3를 만족함을 보인다. 이를 `keygen_prefix_reaches_mu_memory`가 hash 정리의 정확한 술어로 바꾼다.

검증된 정리 4.2 (원시 해시(raw hash) 전제의 비공허성). `generated_raw_mu_preconditions_from_keygen_prefix`는 $base = 0$, $prelen = 0$, 그리고 저장된 검증키 메모리를 택해 no-wrap, raw-branch, valid-region, `sk_memory_prefix` 전제를 동시에 만족시키는 구체적 witness를 구성한다.

이 결과는 전제가 모순이 아니며 내부 KeyGen 반환값에서 도달 가능함을 보인다. 이 witness는 작성한 stores 연산으로 구성된 역사적 로컬 증거다. Week 5의 API-level 정리는 이 witness를 사용하지 않고 실제 caller memory와 생성된 copy 절차만으로 같은 관계에 도달한다.

4.3 3단계: 복사 보조절차(copy helper)에서 실제 원시 KeyGen 호출자까지

`easycrypt/refinement/composition/ApiKeyMemoryBridge.ec`는 focused extraction한 실제 생성 절차들을 직접 대상으로 삼는다. KeyGen의 `__kp_api_copy_2080_to_addr`와 `__kp_api_copy_2752_to_addr`, Sign과 Verify의 `_api_copy_raw_to_2752_prefix`가 그 대상이다.

검증된 정리 4.3 (개별 API 복사 보조절차의 접두부 보존). `keygen_export_vk_mode2_prefix`와 `keygen_export_sk_mode2_prefix`는 종료한 실제 KeyGen exporter가 각각 992바이트 VK와 1408바이트 SK를 raw memory에 기록함을 증명한다. `keygen_export_sk_mode2_prefix_frames_vk`는 최소 disjoint_regions 전제 아래 실제 SK exporter가 기존 VK 접두부를 보존함까지 증명한다. `sign_import_mode2_sk_prefix`와 `verify_import_mode2_vk_prefix`는 유효한 입력 영역에서 실제 importer가 각각 1408바이트와 992바이트를 읽고, 두 로컬 배열의 첫 992바이트가 원래 memory와 일치함을 증명한다. 또한 `sign_verify_api_importer_same_inputs`는 같은 입력에서 두 importer의 반환값과 memory가 같음을 보인다.

순수 adapter 정리 `exported_mode2_regions_agree`, `imported_prefixes_agree`, `imported_sign_sk_reaches_mu_memory`는 export된 두 영역의 접두부 일치에서 Sign/Verify 로컬 접두부 일치와 raw μ memory 전제로 넘어가는 논리적 다리를 제공한다.

`RawApiAddressBridge.ec`의 `canonical_ui64_address_roundtrip`과 `mode2_vk_region_bridge`, `mode2_sk_region_bridge`는 canonical 정수 주소가 `W64.of_int`와 `W64.to_uint`를 거쳐도 같고 양쪽 유효 구간 술어를 만족함을 증명한다. 따라서 이전의 exporter/importer 주소 표현 차이는 명시적 canonical 구간 계약 아래 닫혔다.

`RawApiKeyGenExportComposition.ec`의 `keypair_raw_api_exact_export_trace`는 실제 raw KeyGen caller와 두 copy를 명시한 trace가 최종 memory와 반환값까지 같음을 먼저 고정한다.

검증된 정리 4.4 (실제 원시 KeyGen의 순차 외부 내보내기(export)). `keypair_raw_api_exports_matching_prefixes`는 실제 생성된 `cryptolab_haetae_mode2_keypair_internal`을 연다. 32바이트 seed 읽기, 992바이트 VK와 1408바이트 SK의 canonical 구간, 두 출력의 분리를 전제로 하는 모든 종료 실행에서 반환값이 0이고

$$\text{ExternalKeyPrefix}(\text{mem}_{\text{out}}, \text{sku}, \text{vku})$$

임을 증명한다. 즉 실제 caller에서 먼저 내보낸 VK가 뒤의 SK export에도 보존되고, 두 외부 buffer의 첫 992바이트가 점별로 같다.

이는 helper 사실을 한 caller postcondition으로 합성한 진전이지만 여전히 부분정확성이다. KeyGen retry의 종료는 증명하지 않으며, canonical/valid region은 물리적 메모리 안전성을 대신하지 않는 호출 계약이다.

4.4 4단계: Sign/Verify의 실제 원시(raw) μ 절차

`easycrypt/refinement/composition/ExtractedMuHashPrefix.ec`는 다음 생성 helper를 단계별로 비교한다.

- Sign의 packed-SK absorb와 Verify의 raw VK-memory absorb;
- 같은 pre/message 주소와 길이에 대한 두 raw absorb;
- Keccak state 초기화, permutation, finalize;
- Sign의 64바이트 squeeze와 Verify의 32바이트 squeeze의 앞 32바이트.

그 결과 `sign_verify_raw_mu_top_wrappers_prefix`가 로컬 wrapper 수준의 정 3.16를 증명한다. Week 3의 중요한 진전은 wrapper를 결론 표면에서 제거한 것이다.

검증된 정리 4.5 (실제 생성된 원시 μ 절차의 관계). `sign_verify_generated_raw_mu_prefix`의 양쪽 절차는 각각 실제 생성된 `Sign._sf_mu_rawpre`와 `Verify.__verify_hash_mu`이다. 같은 pre/message 주소와 길이, `RawPreLen`, 두 `ValidRegion`, 992바이트 `SKMemPrefix`, 주소 no-wrap 아래에서

$$\text{MuPrefix}_{32}(\text{res}_S, \text{res}_V)$$

를 증명한다.

Sign 쪽은 `sf_mu_rawpre_refines_raw_top`, Verify 쪽은 `verify_hash_mu_raw_refines_raw_top`으로 wrapper에 정확히 연결한 뒤 중간 정리를 관계적 추이성으로 합성한다. Verify의 raw branch는 별도 flag를 가정하지 않는다. `raw_prelen_shr63_zero`가 실제 `prelen >> 63` 계산이 0임을 증명하여 해당 분기를 선택한다.

4.5 5단계: 도전값 접미부(challenge suffix)의 동일성

Sign challenge transcript는 64바이트의 highbits/LSB 부분 다음에 $\text{take}_{32}(\mu_S^{64})$ 를 흡수하고, Verify도 같은 위치에서 μ_V^{32} 를 흡수한다. 현재 정리는 앞 64바이트 자체가 같다는 것을 아직 증명하지 않고, 같은 challenge state와 위치 64에서 시작한다는 전제를 둔다.

검증된 정리 4.6 (생성된 원시 $\mu \rightarrow$ 도전값 합성). `generated_raw_mu_to_challenge_suffix_zero_loss`는 실제 `_sf_mu_rawpre`와 실제 `__verify_hash_mu`를 각각 실행한 뒤, 실제 Sign/Verify μ_{32} absorb helper를 실행하는 두 합성 프로그램을 비교한다. 검증된 정리 4.5의 전제와 초기 challenge 위치 64 아래에서 반환된 challenge state와 position이 정확히 같다.

`TranscriptBytes.challenge_input_eq_components`는 나중에 highbits와 LSB 바이트까지 같음을 얻으면 전체 입력 리스트가 같아진다는 순수 리스트 보조정리를 제공한다. 따라서 현재 결과와 다음 미해결 의무의 접합면이 명확하다.

4.6 6단계: 실제 원시 Sign/Verify 호출자의 정확 추적(exact trace)

RawApiCallerMuTrace.ec는 실제 cryptolab_haetae_mode2_signature_internal과 ghost 관찰값을 추가한 Sign trace를 정확히 비교한다. sign_raw_api_exact_mu_trace는 최종 memory와 반환값을 보존하고, sign_raw_api_trace_imports_mode2_sk와 sign_raw_api_trace_binds_hash_inputs는 import된 992바이트 접두부와 실제 pre/message pointer 및 길이를 기록한다.

Verify 쪽 RawApiVerifyMuTrace.ec는 full M23, mode 2, internal, raw, cryptolab wrapper를 각각 trace에 정확히 연결한다. 중요한 점은 Verify가 hash 전에 거절할 수 있다는 것이다.

검증된 정리 4.7 (Verify 성공은 실제 해시 꼬리(hash tail)에 도달한다). verify_raw_api_actual_accept_implies_trace_tail_reached와 verify_cryptolab_actual_accept_implies_trace_tail_reached는 실제 반환값이 0인 경우 trace의 tail_reached가 참임을 증명한다. verify_raw_api_actual_accept_binds_hash_inputs와 cryptolab 대응 정리는 그 성공 경로에서 실제 __verify_hash_mu에 들어간 VK/pre/message pointer와 길이를 caller 인자에 묶는다.

이것은 임의 서명의 tail 도달이나 정당한 Sign 출력의 accept를 증명한 것이 아니다. 거절 실행에는 hash 도달이 필요 없고, 정당한 출력의 tail 도달은 별도 기능정확성 의무다.

4.7 7단계: 승인 경로 생성 어댑터(accepted-path generated adapter)와 남은 간극

RawApiMuReachability.ec의 raw_api_region_reaches_mu_zero_loss는 실제 external prefix, Sign import, canonical VK 구간에서 정의 3.3를 구성한다. 여기에는 인위적인 storesmem0 witness가 없다.

검증된 정리 4.8 (승인 추적이 생성 해시/접미부 정리를 인스턴스화함). raw_api_accepting_execution_hash_mu_zero_loss는 Sign trace와 성공한 Verify trace가 공급하는 관찰값 아래 실제 생성 _sf_mu_rawpre와 __verify_hash_mu의 결과에 MuPrefix₃₂가 성립함을 증명한다. raw_api_accepting_execution_generated_challenge_suffix_zero_loss는 같은 전제와 challenge 위치 64에서 실제 생성 suffix helper 결과까지 같게 만든다.

그러나 이 두 정리의 프로그램 표면은 trace module 자체가 아니라 생성 hash와 suffix helper다. Week 6은 아래와 같이 Sign-output frame과 region-local memory 관계를 담았지만, completed trace의 저장 결과로 관계를 운반하는 별도 의미론 간선은 여전히 필요하다.

4.8 8단계: Sign 출력 프레임(Sign-output frame)과 구간 국소 순차 추적(region-local sequential trace)

RawApiSignOutputFrame.ec는 실제 출력 copy와 raw Sign caller를 연다. sign_output_copy_exact_and_frames_reused는 1474바이트 signature 접두부를 정확히 쓰면서 그 바깥 바이트를 frame하고, sign_raw_api_frames_reused_regions는 실제 cryptolab_haetae_mode2_signature_internal까지 이를 올린다.

검증된 정리 4.9 (실제 Sign 출력의 재사용 구간 프레임). 유효한 signature/siglen 출력 구간이 VK, SK, pre, message 구간과 명시적으로 분리되어 있으면, 모든 종료 실행에서 Sign 결과는 0이고 VK 992바이트, SK 1408바이트, pre/message 구간은 보존되며 siglen에는 1474가 기록된다. sign_raw_api_preserves_external_key_prefix는 KeyGen이 만든 ExternalKeyPrefix까지 실제 Sign 호출 뒤로 운반한다. 이 disjointness 계약에는 구체적 witness가 있지만 signing loop의 종료는 결론이 아니다.

RegionLocalMuEquivalence.ec와 RegionLocalMuTop.ec는 전체 memory 등식 요구를 제거한다.

검증된 정리 4.10 (실제 생성 해시의 구간 국소 동치). `sign_verify_generated_raw_mu_prefix_regionwise`는 실제 `_sf_mu_rawpre`와 `__verify_hash_mu`를 비교한다. `mode-2 SK/VK` 접두부 관계, `raw branch`와 `canonical/no-wrap` 구간, 그리고 `pre/message read region`의 **정의 3.5**만으로 $\text{MuPrefix}_{32}(\text{res}_S, \text{res}_V)$ 를 증명한다. 서로 관련 없는 memory 바이트가 같을 필요는 없다.

마지막으로 `RawApiDirectObservedMu.ec`의 `raw_sign_then_verify_actual_exact_trace`는 실제 `raw Sign` 다음 실제 `cryptolab Verify`를 실행하는 프로그램과 두 trace의 순차 실행이 양쪽 결과와 최종 memory를 보존함을 증명한다. `sign_then_verify_trace_accept_implies_tail_reached`와 `sign_then_verify_accept_binds_observed_inputs`는 성공한 `Verify`의 tail 도달과 두 hash descriptor를 사후조건으로 제공한다.

이제 `frame`, `byte locality`, `exact sequence`, `accept control`과 입력 바인딩은 모두 컴파일된다. 남은 **OBL-DIRECT-OBSERVED-MU**는 그럼에도 `SignRawApiMuTrace.observed_mu`와 `VerifyCryptolabMuTrace.observed_mu`를 직접 비교하지 못한다. 기존 `pRHL hash-return` 정리를 서로 다른 residual continuation을 가진 completed trace의 ghost 결과로 옮기는 **OBL-MU-PRODUCT-REPLAY**가 필요하며, Week 7은 새 axiom이나 wrapper widening 없이 이를 만들지 못해 **DEFERRED**로 동결했다. 따라서 $\delta_{\mu, \text{raw_api_accept}} = 0$ 은 여전히 기록하지 않는다.

4.9 9단계: 실제 mode-2 서명 접두부 코덱(signature prefix codec)

Week 7의 기능정확성 경로는 실제 추출된 `_pack_sig_prefix`와 `_unpack_sig_prefix`를 대상으로 전환했다. `Mode2SignaturePrefixPack.ec`의 `pack_sig_prefix_mode2_layout`은 256 challenge bit가 앞 32바이트에, 1024 low coefficient의 하위 signed byte가 다음 1024바이트에 놓임을 증명한다. `Mode2SignaturePrefixUnpack.ec`의 `unpack_sig_prefix_mode2_layout`은 그 배치를 역으로 읽고 사용하지 않는 low-array [1024, 2048)를 보존한다. `Mode2SignaturePrefixRoundTrip.ec`는 두 절차를 순서대로 호출하는 actual-procedure harness를 둔다.

검증된 정리 4.11 (서명 접두부 코덱). 정리 `pack_unpack_sig_prefix_mode2_roundtrip`은 두 실제 생성 절차를 순서대로 호출한다. **정의 3.6** 아래에서 decoded challenge 256개와 mode-2 low 1024개가 입력과 같고, decoded low의 나머지 1024개 word가 보존됨을 증명한다. all-zero 배열이 전제의 비공허성을 보인다.

이는 1056바이트 prefix의 Hoare 부분정확성이다. 1056 이후의 size metadata, hzb/h rANS payload, canonical zero padding, pack output tail과 challenge-array tail frame은 포함하지 않으며, 전체 1474바이트 codec이나 $\delta_{\text{Encoding}} = 0$ 을 결론내리지 않는다.

4.10 10단계: 실제 HBZ 잎(leaf), 표 인증서(table certificate)와 순수 rANS 단계

Week 8은 **OBL-SIG-HBZ-ENCODE-DECODE**를 한 번에 inlining하지 않고 coefficient/symbol leaf, table, rANS core, full wrapper의 네 층으로 분해했다. `Mode2HbzCodecSpec.ec`는 mode-2 상수

$$\text{count} = 1024, \quad m = 13, \quad \text{offset} = 6, \quad \text{scale} = 2^{10}, \quad L = 2^{23}$$

와 **정의 3.7**의 술어를 고정한다.

검증된 정리 4.12 (실제 HBZ 준비/적용(prepare/apply) 잎 역함수). `encode_hb_z1_prepare_mode2_correct`는 실제 `encode_hb_z1_prepare_jazz`가 canonical 계수 $h_i \in [-6, 7)$ 를 symbol $h_i + 6$ 으로 만들고 bad word를 0으로 두며 symbol tail을 보존함을 증명한다. `decode_hb_z1_apply_mode2_correct`는 실제 `apply` 절차가 같은 symbol에서 원래 32비트 계수를 복원하고 coefficient tail을 보존함을 증명한다. 두 actual wrapper를 순서대로 실행하는 `encode_prepare_decode_apply_mode2_inverse`는 첫 1024개 계수의 정확한 복원을 결론낸다. all-zero HBZ 배열이 canonical 전제의 비공허성을 보인다.

이 정리는 prepare/apply의 local inverse이며 rANS buffer, encoder size, full wrapper는 프로그램 표면에 없다. 별도의 Mode2HbzActualBoundary.ec는 focused Week 8 extraction과 Week 7의 signature pack/unpack extraction 안에 있는 실제 _encode_hb_z1_full/_decode_hb_z1_full body가 같은 procedure임을 exact equivalence로 고정한다. 따라서 leaf 정리와 전체 packer가 서로 다른 코드 복사본을 가리키는 문제는 없지만, procedure identity 자체가 inverse를 주지는 않는다.

검증된 정리 4.13 (실제 표 인증서와 순수 rANS 역함수). Mode2HbzTableCertificate.ec는 13개 빈도가 양수이고 합이 1024이며 interval이 $[0, 1024)$ 를 분할함을 보인다. reciprocal multiply/high-half/shift와 bias로 계산한 actual fast encoder 식이 허용 상태 범위에서 수학적 E_s 와 같고 32/64비트 overflow가 없음을 증명한다. Mode2HbzSymbolWordsGenerated.ec의 actual_mode2_hbz_tables_certified는 실제 pack/unpack의 literal encoder/decoder/symbol 배열에 정의 3.8를 적용한다. 마지막으로 Mode2RansCore.ec의 pure_rans_step_inverse와 hbz_fast_step_decode_inverse는

$$D_s(E_s(x)) = x, \quad 0 \leq s < 13, \quad 1 \leq x < \text{hbz_xmax}(s)$$

를 증명한다. $s = 0, x = 1$ 이 fast-step 전제의 구체적 witness다.

4.11 11단계: rANS 바이트 스택(byte stack), 실제 접미부 복사와 결합 제어(control harness)

Week 9은 인코더와 디코더를 곧바로 동기 진행(lockstep)시키지 않고 정의 3.9의 공통 $xs/cuts/bytes$ 의미를 먼저 고정했다.

검증된 정리 4.14 (순수 정규화 추적(normalization trace)과 단어 산술). Mode2RansByteStack.ec의 renorm_len_le2와 renorm_reduced_bounds는 $2^{23} \leq x < 2^{31}$ 에서 각 기호가 0, 1, 2바이트만 방출하고 축소 상태가 $1 \leq r < \text{hbz_xmax}(s)$ 임을 보인다. renorm_bytes_readback은 디코더 순서 구간을 다시 읽으면 정규화 전 상태를 얻음을, serialize32_parse_inverse는 첫 4바이트 리틀엔디언 상태의 왕복을 증명한다. normalized_fast_step_state_bounds와 encode_trace_state_bounds는 정준 추적의 모든 경계 상태가 $[2^{23}, 2^{31})$ 에 머무를 보이고, valid_rans_trace_canonical은 추적 객체가 정의로부터 구성됨을 증명한다. 별도의 Mode2RansNormalization.ec는 실제 W32 이동, 하위 바이트 절단, 디코더 이동/논리합 및 W64 커서 감소를 같은 정수식에 연결한다. 특히 encoder_two_byte_decoder_order는 역방향 저장이 만드는 순방향 구간이 $[\text{high}, \text{low}]$ 임을 검증한다.

이 정리들은 실제 생성 인코더/디코더를 호출하지 않는 순수/단어 앞이다. 따라서 valid_rans_trace를 실제 출력의 사전조건으로 넣거나 디코더 성공을 가정하는 근거로 사용하지 않는다.

검증된 정리 4.15 (실제 접미부 복사와 인코더 결과 분기). copy_encoded_suffix_correct는 실제 __copy_encoded_suffix에 대해

$$0 \leq \text{off}_0, \quad 0 \leq n, \quad \text{off}_0 + n \leq 2048$$

이면 $0 \leq j < n$ 에서 $\text{out}[j] = \text{enc}[\text{off}_0 + j]$ 이고 $[n, 2048)$ 의 기존 out 꼬리가 보존됨을 증명한다. 구성된 디코더 버퍼를 쓰지 않고 실제 복사 반복문을 연 Hoare 부분정확성 정리다.

actual_rans_encode_mode2_control과 actual_rans_decode_mode2_control 중 인코더 정리는 구체적 mode-2 표/개수와 정준 기호열을 고정하되 초기 상태는 임의로 두고, 디코더 정리는 구체적 표와 mode2_decode_state 입력을 고정한다. 두 정리는 각각 반환 $\text{bad} \in \{0, 1\}$ 만 보인다. 이어 정의 3.15의 actual_rans_harness_branches_on_encoder_result는 실제 인코더-복사-디코더 순서, 실패 시 디코더 미실행, 성공 분기의 정확한 디코더 입력 필드를 증명한다. 이 결론에는 추적 등식, 디코더 성공이나 기호 왕복이 없다.

Week 9 종료 시 정확한 상태는 다음과 같다.

의무	상태
HBZ 준비/적용, 앞 역함수와 유계 꼬리 프레임	PROVED
집중/전체 실제 절차 경계	PROVED
실제 13기호 표 인증서	PROVED
순수 및 역수 빠른 단일 단계 역함수	PROVED
순수 정규화 바이트 역함수와 상태 범위	PROVED
실제 접미부 복사의 점별/프레임 정리	PROVED
실제 인코더-복사-디코더 결합 제어	PROVED
실제 인코더 외부 반복문 추적 정제	PARTIAL
실제 디코더 추적 소비 정제	PARTIAL
실제 rANS 핵심 역함수와 성공 증인	PARTIAL
성공 조건부 전체 HBZ 역함수	PARTIAL

실제 인코더는 기호를 역순으로 처리하면서 정규화 바이트를 뒤쪽 커서에 쌓고, 디코더는 이를 앞에서부터 소비한다. Week 9에는 이 역방향 스택/순방향 소비 관계를 실제 $(i, x, off, buffer)$ 와 순수 (x_j, c_j, B) 사이에 세우는 두 외부 불변식이 없었다. 따라서 [경계 명제 3.1](#)에 따라 정준 HBZ 만으로 인코더 성공을 가정하지 않았고, **OBL-SIG-HBZ-ENCODE-DECODE**는 **PARTIAL**이었다. 이 표와 CONTINUE-RANS는 Week 9 종료 시점의 역사적 판정이다.

4.12 12단계: 실제 인코더 내부 의미와 성공 시 크기

Week 10은 디코더로 이동하지 않고 실제 인코더 경계를 더 세분화했다. Mode2RansArrayList Bridge.ec는 [정의 3.10](#)의 배열/목록 접미부와 구간/프레임 연산을 정의하고, 1024번째 접미부 기저식, 한 원소 점화식, 정준성, 이어 붙이기와 한 기호 encode_trace 점화식을 증명한다.

검증된 정리 4.16 (배열/목록 접미부와 실제 단어 단계). encode_trace_suffix_extension은 배열의 i 번째 기호와 $i + 1$ 번째 접미부에서 한 기호 추적을 구성한다. Mode2RansEncoder WordStep.ec의 actual_mode2_encoder_word_step_correct는

$$0 \leq s < 13, \quad 1 \leq \text{uint}(x) < \text{hbz_xmax}(s)$$

에서 실제 mode-2 표의 역수 곱, 이동과 편향으로 계산한 W32 결과가 순수 hbz_fast_encode_step의 W32 표현과 같음을 증명한다. 이것은 **OBL-RANS-ENCODER-WORD-STEP**의 단어 수준 정리이며, 그 자체로 생성된 외부 반복문의 Hoare 정리는 아니다.

Mode2RansEncoderInnerProgress.ec는 순수 정규화 길이가 0, 1, 2임과 매 단계의 나눗셈/바이트 추가를 증명한다. 실제 생성 반복문의 재사용 술어는 Mode2RansEncoderTrace.ec에 있고, 직접 Hoare 증명은 Mode2RansEncoderActualInner.ec에 있다.

검증된 정리 4.17 (실제 생성 내부 반복문의 바이트/프레임 단계). actual_rans_encode_inner_no_underflow는 실제 RansEncodeTarget.M._rans_encode를 열어 [정의 3.11](#)의 국소 단계 불변식을 유지한다. 실제 전이는

$$off \leftarrow off - 1; \quad \text{encp}[off] \leftarrow \text{low8}(x); \quad x \leftarrow x \gg 8$$

이며, 쓴 바이트 구간과 더 낮은 접두부 프레임을 보존하고 W64 감소가 언더플로하지 않음을 검증한다. 절차의 반환 사후조건은

$$\text{bad}' \neq 0 \quad \vee \quad (\text{bad}' = 0 \wedge \text{uint}(off') \leq 1020)$$

이다. 현재 위상 범위 $0 \leq k \leq 4$ 는 보수적이다. 실제 위상을 순수 0/1/2 정규화 바이트와 동일 시하고, 내부 호출 전 배열을 누적 외부 추적에 연결하는 부분은 **OBL-RANS-ACTUAL-INNER-NORMALIZE**에 **PARTIAL**로 남는다.

검증된 정리 4.18(최종 직렬화 앞과 실제 성공 시 크기). `Mode2RansEncoderSerialization.ec`는 네 번의 실제 패턴 `set8` 쓰기가 `serialize32_le`와 같고 배열 밖 및 앞쪽 프레임을 보존함을 증명한다. 이는 직렬화 앞 정리이며 아직 생성 절차의 전체 사후조건으로 합성되지 않았다.

별도로 `actual_rans_encode_success_size_bound`는 실제 `_rans_encode`의 실패 분기를 유지하면서 성공 시

$$0 \leq \text{uint}(\text{off}') \leq 1020, \quad 4 \leq 1024 - \text{uint}(\text{off}') \leq 1024$$

를 증명한다. 이는 **OBL-RANS-ENCODER-SUCCESS-SIZE**의 실제 절차 부분정확성이다. 성공의 도달 가능성이나 종료를 증명하지 않는다.

Week 10 종료 시의 정확한 상태는 다음과 같다.

의무	상태
OBL-RANS-ARRAY-LIST-BRIDGE	PROVED
OBL-RANS-ENCODER-WORD-STEP	PROVED (단어 수준)
OBL-RANS-ACTUAL-INNER-NORMALIZE	PARTIAL
OBL-RANS-FINAL-SERIALIZATION	PROVED (앞 정리)
OBL-RANS-ENCODER-SUCCESS-SIZE	PROVED (실제 부분정확성)
OBL-RANS-ENCODE-REFINEMENT	PARTIAL
OBL-RANS-ACTUAL-SUCCESS-WITNESS	PARTIAL
OBL-RANS-DECODE-REFINEMENT 과 핵심 역함수	PARTIAL
OBL-SIG-HBZ-ENCODE-DECODE	PARTIAL

남은 인코더 사후조건은 실제 반환 접미부가 `symbol_list_of_array(symbols0)`에 적용한 `trace_bytes`와 점별로 같고, 그 길이와 반환 오프셋의 합이 1024이며, 쓰지 않은 접두부가 프레임됨을 한꺼번에 말해야 한다. 이를 위해 외부 불변식에 $0 \leq i \leq 1024$, 접미부 추적의 상태, 오프셋-길이 등식, `segment_matches`, `prefix_frame`을 함께 유지해야 한다. 한 외부 반복의 실제 표 읽기/보호/역수 이동 제어를 이미 증명된 내부 바이트 단계와 단어 단계에 연결하고, 마지막 네 저장을 누적 접미부와 합성하는 간선이 Week 10에는 없었다.

따라서 Week 10의 역사적 판정은 **CONTINUE-ENCODER**였고, Week 11은 이 단일 간선만을 대상으로 삼았다.

4.13 13단계: Week 11 실제 인코더 추적 폐쇄

Week 11은 Week 10의 국소 내부 불변식에 순수 기호 꼬리(`pure symbol tail`)의 추적을 직접 붙인 다음 정확한 불변식을 설치했다. 편의를 위해

$$\begin{aligned} \ell &:= \text{mode2_normalization_len}(x_0, s), \\ W_k &:= \text{inner_written_suffix}(x_0, s, k), \quad B_t := \text{encode_trace}(\text{tail}).\text{bytes} \end{aligned}$$

라 쓰면 핵심 형태는 다음과 같다.

$$\begin{aligned} I_{\text{tail}}(k) \equiv & 0 \leq k \leq \ell \\ & \wedge \text{segment_matches}(\text{encp}, \text{off}_0 - k, W_k ++ B_t) \\ & \wedge \text{prefix_frame}(\text{enc}_0, \text{encp}, \text{off}_0 - k). \end{aligned}$$

실제 가드가 거짓이 되는 지점에서 $k = \text{mode2_normalization_len}(x_0, s) \leq 2$ 가 도출된다. 생성 표의 $x_{\max}$, 역수, 편향, 패킹된 보수/이동값과 이동 사다리는 한 번의 전용 단어 연결을 거쳐 순수 빠른 단계와 같아진다.

Week 11 파일의 역할은 다음과 같다. 아래 파일명에는 공통 접두부 Mode2RansEncoder를 붙여 읽는다.

- TailInvariant.ec: 불변식의 초기화, 한 바이트 진행, 정확한 종료와 외부 한 기호 성공 전이;
- GeneratedWordStep.ec: 생성 표 읽기와 단어 갱신의 연결;
- SerializationComposition.ec, Finalization.ec, GeneratedFinalization.ec: 직렬화된 상태를 누적 추적 앞에 붙이는 실제 최종 저장 합성;
- ActualTraceClosure.ec: 실제 내부/외부 반복과 최종 저장을 직접 여는 Hoare 정리;
- OuterRefinement.ec: 전체 성공 접미부를 펼쳐 쓴 최종 따름정리(corollary).

검증된 정리 4.19 (실제 인코더의 정확한 성공 접미부). `actual_rans_encode_trace_closure`는 실제 생성 `RansEncodeTarget.M._rans_encode`를 직접 열어 다음 실패/성공 사후조건을 증명한다. 아래에서

$$B_0 := \text{trace_bytes}(\text{symbol_list_of_array}(\text{symbols}_0))$$

로 둔다.

$$\begin{aligned} \text{bad}' \neq 0 \quad \vee \quad & (\text{bad}' = 0 \wedge 0 \leq \text{off}' \leq 1020 \wedge 4 \leq 1024 - \text{off}' \leq 1024 \\ & \wedge \text{segment_matches}(\text{encp}', \text{off}', B_0) \\ & \wedge \text{off}' + |B_0| = 1024 \\ & \wedge \text{prefix_frame}(\text{enc}_0, \text{encp}', \text{off}')). \end{aligned}$$

성공은 사전조건이 아니며 추적은 실제 입력 기호 배열에서 정준적으로 계산된다. `actual_rans_encode_trace_refinement`가 같은 결론을 펼쳐 쓴 최종 따름정리다. 두 정리는 57대상-no-eco 전체 검증에서 컴파일됐다.

Week 11 종료 상태는 다음과 같다.

의무	상태
OBL-RANS-ACTUAL-INNER-NORMALIZE	PROVED
OBL-RANS-FINAL-SERIALIZATION	PROVED (실제 합성)
OBL-RANS-ENCODE-REFINEMENT	PROVED (성공 조건부 부분정확성)
OBL-RANS-ACTUAL-SUCCESS-WITNESS	PARTIAL
OBL-RANS-DECODE-REFINEMENT	PARTIAL
OBL-RANS-CORE-INVERSE	PARTIAL
OBL-SIG-HBZ-ENCODE-DECODE	PARTIAL

Week 11 종료 판정은 GO-ENCODER였다. 이는 인코더 정제 간선이 닫혀 Week 12가 실제 디코더 추적 정제로 이동할 수 있다는 뜻이지, 인코더 종료나 성공 분기 도달, 디코더 성공, HBZ 왕복 또는 인코딩 무손실이 증명됐다는 뜻은 아니었다.

4.14 14단계: Week 12 실제 디코더 의미 정제

Week 12는 순수 추적의 존재를 디코더 성공으로 취급하지 않고, 실제 생성 RansDecodeTarget.M._rans_decode의 두 반복문을 직접 열었다. 파일명은 공통 접두부 Mode2RansDecoder를 붙여 읽는다.

- Cursor.ec, CursorSteps.ec: 참조 상태, 구간과 커서의 시작/한 단계/최종 경계 및 읽기 범위;
- WordStep.ec, ActualWord.ec, GeneratedStep.ec: 구체적인 symbol_words 조회, dsyms_words 필드와 실제 W32/W64 갱신을 수학적 디코더 단계에 연결;
- Normalization.ec: 부분 바이트 재생과 0/1/2바이트 가드의 정확성;
- ActualTrace.ec: 실제 외부/내부 반복 불변식, 초기 파싱, 한 기호 전이와 정확한 출구;
- TopHoare.ec: 실제 생성 절차를 직접 대상으로 하는 최상위 Hoare 정리.

예상 기호 목록 s 에 대한 참조 경계는

$$c_0 = 4, \quad c_{i+1} = c_i + |\beta_i|, \quad c_{1024} = |\text{trace_bytes}(s)|, \\ x_0 = \text{ParseLE32}(\text{trace_bytes}(s)[0..4)), \quad x_{1024} = 2^{23}.$$

generated_decoder_parse32_from_trace가 첫 식의 실제 4바이트 파싱을 연결하고, 생성 표 정리들은 $x_i \bmod 1024$ 가 선택한 기호와 환원 상태를 수학적 단계에 맞춘다. 내부 불변식은

$$k = \text{uint}(\text{off}) - c_i, \quad x = \text{decoder_replay_prefix}(x_{i+1}, s_i, k)$$

를 유지한다. 생성 가드는 $k < |\beta_i|$ 와 같고 $|\beta_i| \leq 2$ 이므로 실제 반복은 정확히 0, 1 또는 2바이트를 소비한다.

검증된 정리 4.20 (실제 디코더의 정확 추적 부분정확성). actual_rans_decode_trace_refinement의 사전조건은 정준 1024기호 배열, 그 배열에서 계산한 정확한 trace_bytes, 크기 $4 \leq n \leq 1024$, 상태 필드 (1024, n , 13), 실제 mode-2 두 디코더 표와 점별 추적 읽기 관계다. 성공이나 출력 등식은 사전조건에 없다. 모든 종료 실행의 사후조건은

$$\text{bad}' = 0, \quad \text{off}' = n, \\ \text{decoded}'[0..1024) = \text{expected}[0..1024), \quad \text{decoded}'[1024..2048) = \text{decoded}_0[1024..2048)$$

이다. 내부 출구의 $x = 2^{23}$ 은 실제 최종 거절 검사를 닫는 데 사용되지만 반환 ABI에는 노출되지 않는다. 이 정리는 실제 생성 절차를 직접 대상으로 하며 래퍼나 순수 디코더 정리가 아니다.

Week 12 종료 상태는 다음과 같다.

의무	상태
디코더 커서/표/단어 단계 연결	PROVED
OBL-RANS-DEC-NORMALIZE	PROVED (실제 반복문 연결)
OBL-RANS-DECODE-REFINEMENT	PROVED (정확 추적 부분정확성)
OBL-RANS-CORE-INVERSE	PARTIAL
OBL-RANS-ACTUAL-SUCCESS-WITNESS	PARTIAL
OBL-SIG-HBZ-ENCODE-DECODE	PARTIAL

65대상 -no-eco 전체 검증이 통과했고 Week 12 당시 판정은 GO-DECODER다. Week 13의 단일 간선은 실제 Mode2RansActualHarness.run 안에서 인코더의 segment_matches와 복사 보조절차의 slice_eq를 디코더의 점별 정확 추적 입력으로 운반하는 합성이다. 이 결합, 실제 인코더 성공 도달, 두 반복문의 종료, 전체 HBZ 래퍼와 인코딩 무손실은 Week 12 정리에서 따라오지 않는다.

4.15 15단계: Week 13 실제 rANS 핵심 역함수 합성

Week 13은 Week 12의 마지막 미연결 간선을 두 작성 이론으로 닫았다. Mode2RansCore CompositionBridge.ec는 다음 사후조건 어댑터를 제공한다.

- `copied_suffix_is_exact_trace`: 실제 인코더의 `segment_matches`와 실제 복사의 `slice_eq`에서 복사 버퍼 오프셋 0의 정확 추적을 도출한다.
- `segment_matches_implies_exact_decoder_segment_input`: 전역 추적 구간을 각 정규화 구간의 점별 디코더 읽기로 펼친다.
- `actual_decoder_input_from_configured_trace`: 실제 세 상태 저장과 복사된 추적으로 Week 12의 전체 디코더 입력 술어를 구성한다.
- `encoder_success_size_word_bridge`: 실제 성공 오프셋 범위로 W64 뿔셈의 무랩(no-wrap)과 정수 추적 크기 등식을 증명한다.

검증된 정리 4.21 (세 실제 절차의 성공 조건부 핵심 역함수). Mode2RansCoreActualInverse.ec의 `actual_rans_encode_copy_decode_inverse`는 Mode2RansActualHarness.run을 직접 대상으로 한다. 사전조건은 여섯 초기 인자의 바인딩과 정준 `mode2_hbz_symbol_stream` 뿐이며 인코더 성공을 요구하지 않는다. 모든 종료 실행에서 다음 두 분기 중 하나가 성립한다.

$$\begin{aligned} bad_{enc} \neq 0 &\implies \neg decoder_ran \wedge decoded' = decoded_0 \wedge state' = state_0, \\ bad_{enc} = 0 &\implies decoder_ran \wedge bad_{dec} = 0 \wedge off_{dec} = encoded_size \\ &\quad \wedge decoded'[0..1024) = symbols[0..1024). \end{aligned}$$

성공 분기는 디코더 입력 필드 (1024, `encoded_size`, 13), 출력 [1024, 2048) 꼬리 프레임, 실제 인코더의 정확한 접미부와 초기 접두부 프레임도 보존한다. 따라서 복사된 추적이나 디코더 결과를 전제로 되풀이하는 순환 가정은 없다.

이 정리의 전제는 정준 all-zero 기호 배열로 만족 가능하지만, 결론이 실제 인코더의 실패/성공 분기이므로 성공 분기의 도달 가능성은 따라오지 않는다. 또한 결합 모듈은 세 고정 생성 절차를 직접 호출하는 증명 작성 harness이며, `production_encode_hb_z1_full/_decode_hb_z1_full` 래퍼 자체는 아니다.

Week 13 완료 상태는 다음과 같다.

의무	상태
복사 추적/점별 디코더 입력/W64 크기 어댑터	PROVED
OBL-RANS-CORE-INVERSE	PROVED (성공 조건부 부분정확성)
OBL-RANS-ACTUAL-SUCCESS-WITNESS	PARTIAL
인코더/디코더 종료	PARTIAL
OBL-SIG-HBZ-ENCODE-DECODE	PARTIAL (실제 전체 래퍼 합성)

67대상 `-no-eco` 전체 검증과 독립 정리 표면 검사가 통과했고 최신 판정은 GO-CORE다. Week 14의 단일 구현 간선은 이미 닫힌 `prepare/apply` 역함수와 이 핵심 역함수를 실제 전체 HBZ 래퍼 경계에서 성공 조건부로 합성하는 것이다. 실제 성공 증인, 종료, h 코덱, 전체 1474 바이트 코덱과 인코딩 무손실은 Week 13 정리에서 따라오지 않는다.

5 보안 관점에서 정확히 무엇을 얻었는가

5.1 국소 무손실(zero loss)의 의미

검증된 정리 4.6는 확률분포 사이의 근사나 random-oracle 이상화를 사용하지 않는다. 같은 전제 아래 두 결정론적 생성 절차의 반환 상태가 같다는 관계적 동치다. 따라서 이 특정 seam에 대해서만

$$\delta_{\mu, \text{raw_top}} = 0$$

이라고 기록할 수 있다.

경계 명제 5.1 (승격할 수 없는 등식). 국소 등식은

$$\delta_{\text{Sign}} = 0, \quad \delta_{\text{Verify}} = 0, \quad \delta_{\text{Encoding}} = 0$$

중 어느 것도 함의하지 않는다. 검증 스크립트는 이러한 부당한 승격 문자열이 작성 트리에 나타나면 실패하도록 되어 있다.

또한 검증된 정리 4.8의 이름에 zero_loss가 들어가도 현재 허용되는 API 등식은 아니다. 교차 호출 프레임(cross-call frame), 구간 국소 해시 관계와 정확 순차 추적은 이제 컴파일되지만, 생성 해시의 반환값을 완료된 추적의 저장된 관찰값으로 운반하는 생성/재전달 결론이 없으므로

$$\delta_{\mu, \text{raw_api_accept}} = 0$$

은 아직 기록하지 않는다.

이 구분은 equation (1)의 손실 항을 과소평가하지 않기 위해 필수적이다. 현재 정리는 δ_{Sign} 와 δ_{Verify} 내부의 한 결정론적 부분을 0으로 만들었을 뿐, sampler, rejection, packing, parsing, public context, API memory를 포함한 전체 하이브리드를 닫지 않았다. 다만 API memory의 많은 하위 경계—주소 왕복, KeyGen 순차 export, Sign trace와 output frame, accepted Verify tail과 입력 바인딩, region-local hash read relation—은 이제 실제 caller 정리로 닫혔다.

Week 8–10의 HBZ 잎/표/단일 단계, 순수 바이트 스택, 실제 접미부 복사와 결합 제어, 그리고 실제 인코더의 내부 바이트/프레임 단계와 성공 시 크기 범위도 결정론적 기능정확성 증거다. Week 11의 전체 실제 인코더 성공 접미부 등식까지 같은 종류의 증거로 닫혔다. Week 12의 실제 디코더 정확 추적 복원도 정확한 입력 관계 아래의 결정론적 부분정확성으로 닫혔다. Week 13은 세 실제 절차의 순차 결합까지 성공 조건부 부분정확성으로 닫았다. 그러나 production 전체 HBZ 래퍼 합성과 실제 성공 증인이 열려 있으므로 이를 $\delta_{\text{Encoding}} = 0$ 으로 승격할 수 없다. 인코더/디코더 종료와 실제 성공 인코딩 증인이 없다는 사실도 부모 정리의 비공허성 경계로 남는다.

5.2 현재 주장 행렬

표 2: 주요 주장과 현재의 정확한 강도

주장	상태	현재 확보된 의미 / 남은 경계
KeyGen 패킹 접두부 (packed prefix)	PROVED	종료한 mode-2 생성 KeyGen 반환값에서 $sk[0..992) = vk[0..992)$; 종료와 분포는 별도
KeyGen→메모리 도달 가능성 (reachability)	PROVED	내부 반환값으로부터 구체적 로컬 Verify 메모리 witness 구성; Week 5 API 정리에는 사용하지 않는 역사적 seam
개별 public API key copy helper	PROVED	실제 생성 exporter/importer의 992/1408-byte 접두부 보존 부분정확성

주장	상태	현재 확보된 의미 / 남은 경계
정준 주소 연결 (canonical address bridge)	PROVED	정수 ABI 주소와 W64 exporter/hash 주소의 왕복 및 992/1408/1474-byte region 특수화
실제 원시 KeyGen 내보내기(actual raw export)	PROVED	실제 caller가 종료하면 분리된 외부 VK/SK 영역의 첫 992바이트가 같음; retry 종료는 별도
실제 원시 Sign 추적 (actual raw trace)	PROVED	실제 caller의 반환값/memory를 보존하며 SK import와 hash pointer/length 관찰값을 노출
실제 Verify 승인 경로 (accepted path)	PROVED	raw/cryptolab exact trace와 <i>accept</i> $\Rightarrow$ <i>tail</i> ; 성공 시 실제 hash 입력 바인딩
원시 보조절차 μ (raw-helper μ)	PROVED	실제 흡수(absorb), Keccak, 마무리(finalize), 64/32 바이트 짜내기(squeeze)의 관계
생성 최상위 제어 (generated top control)	PROVED	실제 <code>_sf_mu_rawpre</code> 와 <code>__verify_hash_mu</code> 원시 분기(raw branch)를 직접 비교
도전값 접미부(challenge suffix)	PROVED	위치 64 이후 실제 μ_{32} absorb 결과 상태와 위치 일치
승인 생성 어댑터 (accepted generated adapters)	PROVED	caller trace 사실이 실제 생성 hash-call μ prefix와 위치-64 suffix helper 정리를 인스턴스화
Sign 출력 프레임 (Sign-output frame)	PROVED	실제 raw Sign의 1474-byte signature와 8-byte siglen 쓰기가 분리된 VK/SK/pre/message 구간을 보존하는 부분정확성
구간 국소 생성 μ (region-local generated μ)	PROVED	전체 memory 등식 없이 실제 VK/pre/message read region만으로 생성 hash 반환값의 prefix 관계를 증명
실제 Sign $\rightarrow$ Verify 순서 (actual sequence)	PROVED	실제 두 caller와 trace 순차 실행의 결과/최종 memory 동치, accept-to-tail 및 hash descriptor 사후 바인딩
승인 원시 API μ 합성 (accepted composition)	PARTIAL	trace의 직접 observed_mu 등식만 남음; frame/locality/control은 닫힘
생성/재전달 운반 (product/replay transport)	DEFERRED	생성 hash return의 pRHL 결과를 completed trace의 stored observed_mu로 옮길 정당한 규칙이 없음
서명 접두부 코덱 (signature prefix codec)	PROVED	실제 pack/unpack의 challenge 32-byte와 low 1024-byte round trip; canonical 입력 아래 부분정확성
서명 접두부 프레임 (signature prefix frame)	PARTIAL	unpack low-array tail은 보존되지만 pack signature tail과 challenge-output tail은 열림
HBZ 준비/적용 앞 (prepare/apply leaf)	PROVED	실제 leaf가 canonical 1024개 계수를 13-symbol alphabet으로 옮겼다가 복원하고 bounded array tail을 보존
HBZ 실제 절차 경계 (actual procedure boundary)	PROVED	focused/full extraction과 실제 signature pack/unpack extraction의 HBZ full procedure가 exact equivalent; inverse는 별도
HBZ 표 인증서(table certificate)	PROVED	실제 encoder/decoder/symbol 배열이 양의 빈도, 1024-slot partition 및 reciprocal arithmetic certificate를 만족
순수 rANS 단일 단계 (pure single step)	PROVED	수학적 $D_s(E_s(x)) = x$ 와 허용 범위의 실제 reciprocal fast-step 일치; normalization byte와 loop는 제외
순수 rANS 바이트 스택 (byte stack)	PROVED	0/1/2-byte normalization readback, little-endian state parse, <i>xs/cuts/bytes</i> construction과 $[2^{23}, 2^{31})$ state-bound 보존; generated loop는 제외

주장	상태	현재 확보된 의미 / 남은 경계
실제 인코딩 접미부 복사 (actual encoded-suffix copy)	PROVED	실제 <code>__copy_encoded_suffix</code> 의 점별 slice equality와 unused-tail frame; Week 13은 actual encoder success에서 입력 정수 범위와 W64 무렵을 유도해 핵심 합성에 사용
실제 rANS 결합 제어 (actual harness control)	PROVED	실제 encoder-copy-decoder 호출과 encoder 반환값에 따른 실패/성공 분기 및 decoder field 초기화; 이 기존 제어 정리 자체의 inverse 결론은 없음
배열/목록 접미부 연결 (array/list suffix bridge)	PROVED	정확한 1024원소 목록/접미부 점화식, 구간/프레임 연산 및 한 기호 추적 확장
실제 인코더 단어 단계 (actual encoder word step)	PROVED	실제 mode-2 표의 W32/W64 역수 계산이 순수 빠른 단계와 같음; 단어 수준 등식
실제 내부 정규화(actual inner normalization)	PROVED	실제 생성 반복문이 순수 꼬리, 정확한 0/1/2바이트, 커서와 초기 접두부 프레임을 보존하고 실제 guard에서 정확히 종료하는 부분정확성
최종 직렬화(final serialization)	PROVED	네 실제 패턴 저장에 누적 정규화 추적 앞에 리틀엔디언 상태를 붙이고 초기 접두부 프레임을 보존하도록 실제 생성 절차 출구에 합성됨
실제 인코더 성공 크기 (actual success size)	PROVED	실제 <code>_rans_encode</code> 의 실패 분기를 보존하며 성공 시 $0 \leq off \leq 1020$, $4 \leq 1024 - off \leq 1024$; 종료/성공 도달은 제외
실제 rANS 인코더 정제 (actual encoder refinement)	PROVED	실제 반환 <code>bad</code> 의 실패/성공 분기; 성공 시 전체 <code>trace_bytes</code> , 정확한 오프셋-길이 등식과 초기 접두부 프레임을 주는 성공 조건부 부분정확성
실제 rANS 디코더 정제 (actual decoder refinement)	PROVED	정준 1024기호의 정확 <code>trace_bytes</code> , 실제 표와 상태 입력 아래 종료한 <code>_rans_decode</code> 가 <code>bad = 0</code> , 정확한 소비 크기, 전체 기호 복원과 출력 꼬리 프레임을 만족; 종료/손실없음은 별도
실제 rANS 핵심 역함수 (actual core inverse)	PROVED	실제 인코더 실패 시 디코더 미실행/초기 상태 보존, 성공 시 실제 복사와 디코더의 <code>bad = 0</code> , 정확한 소비 크기, 1024기호 복원 및 꼬리 프레임을 한 harness에서 증명; 성공 도달과 종료는 별도
실제 전체 HBZ 부모 (actual full-HBZ parent)	PARTIAL	prepare/apply 앞과 실제 rANS 핵심 역함수는 닫혔지만 production full wrapper의 프레임/크기 합성과 성공 증인이 열림
전체 서명 코덱(full signature codec)	BLOCKED	남은 actual full-HBZ wrapper, h rANS inverse, metadata, padding, residual frame과 canonical parse가 필요
정당한 Sign 출력 꼬리 도달(legitimate tail reach)	PARTIAL	prefix inverse만 닫혔고 full codec의 rANS core 층까지 진전; full-HBZ wrapper, norm, hint, response/arithmetic가 필요한 별도 기능정확성 의무
공용 문맥 μ (public-context μ)	SPECIFIED	문맥 길이 바이트, 문맥, Verify 상위 비트 분기 (high-bit branch)의 일치 정리 없음
전체 도전값 호출(full challenge call)	PARTIAL	μ 접미부와 목록 합동성(list congruence)은 있으나 highbits/LSB, 패딩/도메인(padding/domain), 짜내기/표본기(squeeze/sampler)가 열림
전체 KeyGen/Sign/Verify FC	SPECIFIED	abstract goal은 있으나 세 API의 paper refinement 정리 없음
구현 EUF-CMA	SPECIFIED	모든 API loss, query accounting, 정확한 paper/ROM instance가 필요
논문 보안 reduction	PARTIAL	기존 조건부 chain은 있으나 정확한 transcript/distribution/forking 인스턴스가 열림

5.3 가정, 계약, 증명 의무의 구분

최종 보안 정리에 남길 수 있는 암호학적 가정은 구체적인 HAETAE-2 MLWE, BST-MSIS, ROM 게임뿐이다. 반면 다음 항목은 암호학적 가정으로 숨기면 안 된다.

- public buffer의 유효성, 크기, 분리, alias 조건;
- canonical 길이와 mode-2 상수;
- retry의 종료/losslessness와 RNG/random-tape 해석;
- FIPS202 바이트 transcript, padding, domain separation, endianness;
- hyperball sampler와 accepted-signature 분포;
- FFT 고정소수점 오차와 tied-minimum 정책.

이들은 API refinement, 확률 프로그램, 수치해석에서 각각 증명해야 할 계약이다. 현재 작성 이론에는 새 axiom이나 proof hole을 두어 이 간극을 덮지 않는다.

6 남은 증명 의무와 권장 진행 순서

6.1 Week 5/6이 담은 실제 API 경계

Week 5는 실제 caller의 control과 입력을, Week 6은 cross-call memory 조건을 다음 범위에서 달았다.

- RawApiAddressBridge가 canonical 정수 주소와 W64 주소의 왕복을 증명한다.
- keypair_raw_api_exports_matching_prefixes가 실제 raw KeyGen의 두 순차 export 뒤 외부 VK/SK의 첫 992바이트 일치를 증명한다.
- exact Sign/Verify trace, *accept* $\Rightarrow$ *tail*, accepted hash-input binding이 실제 caller의 control과 hash 인자를 노출한다.
- 실제 Sign의 signature/siglen 출력은 명시적으로 분리된 VK/SK/pre/message 구간을 보존하고 KeyGen의 외부 key prefix를 유지한다.
- 실제 생성 hash는 전체 memory 등식 대신 VK/pre/message read region의 바이트 등식만으로 비교된다.
- 실제 raw Sign 다음 cryptolab Verify의 실행과 두 trace의 순차 실행은 결과와 최종 memory까지 정확히 대응한다.

raw_api_region_reaches_mu_zero_loss는 이 실제 external-memory 사실과 Sign import를 sk_memory_prefix로 바꾼다. 이 경로는 검증된 정리 4.2의 constructed stores를 사용하지 않는다.

6.2 동결된 보안 관문(gate): 저장 μ 의 생성/재전달(product/replay)

이전의 **OBL-API-KEY-MEMORY-RAW**는 구현의 early-reject 제어흐름보다 강한 무조건 명제였기 때문에 Week 5에서 분해되었다. Week 6 이후 직접 잔여 의무는 다음 하나의 의미론 간선으로 좁혀졌다.

미해결 의무 6.1 (OBL-DIRECT-OBSERVED-MU). 실제 raw Sign trace와 성공한 cryptolab Verify trace의 순차 실행 뒤

$$\text{MuPrefix}_{32}(\mu_{S,\text{trace}}, \mu_{V,\text{trace}})$$

를 직접 결론으로 얻는다. 여기서 두 항은 각각 `SignRawApiMuTrace.observed_mu`와 `VerifyCryptolabMuTrace.observed_mu`다. 생성 hash return의 pRHL 정리를 이미 완료된 두 trace의 ghost field에 운반할 적당한 product/replay 규칙이 필요하다.

검증된 정리 4.9과 검증된 정리 4.10, actual sequential trace는 frame, locality, control 및 input binding을 이미 공급한다. 하지만 Sign hash 뒤 signing continuation과 Verify hash 뒤 challenge/compare continuation이 서로 달라, 열린 signing losslessness 없이 한쪽 continuation을 버릴 수 없다. Week 7은 추가 observation wrapper, 목표를 반복하는 precondition, SHAKE locality axiom을 거부하고 **OBL-MU-PRODUCT-REPLAY**를 **DEFERRED**인 명시적 paper boundary로 동결했다. 따라서 이 lane에서 $\delta_{\mu, \text{raw_api_accept}} = 0$ 을 기록하지 않으며 같은 seam에 wrapper를 더 쌓지 않는다.

6.3 최근 닫힌 관문과 현재 진행 관문: 실제 rANS 핵심에서 전체 HBZ로

Week 7-11은 정준 서명 접두부, HBZ 앞/표, 순수 바이트 추적, 실제 접미부 복사와 결합 제어, 그리고 실제 인코더의 성공 접미부 정제를 차례로 닫았다. Week 12는 **OBL-RANS-DECODE-REFINEMENT**를 정확 추적 부분정확성으로 닫았다. Week 13은 이 세 결과를 `Mode2RansActualHarness.run` 안에서 합성해 **OBL-RANS-CORE-INVERSE**를 성공 조건부 Hoare 부분정확성으로 닫았다.

`actual_rans_encode_copy_decode_inverse`의 전제는 초기 인자 바인딩과 `mode2_hbz_symbol_stream`뿐이다. 인코더가 실제로 실패하면 복사와 디코더를 실행하지 않고 초기 디코더 배열/상태를 유지한다. 인코더가 성공하면 실제 복사 구간에서 Week 12의 정확 추적 입력 전체를 구성하여

$$\begin{aligned} \text{bad}_{\text{dec}} &= 0, & \text{off}_{\text{dec}} &= \text{encoded_size}, \\ \text{decoded}[0..1024) &= \text{expected}[0..1024), & \text{decoded}[1024..2048) &= \text{decoded}_0[1024..2048) \end{aligned}$$

을 얻는다. 복사된 추적, 디코더 점별 읽기, 디코더 성공과 출력 등식은 사전조건이 아니다. `encoder_success_size_word_bridge`도 실제 성공 범위에서 W64 크기 뺄셈의 무렵을 도출하므로 임의의 정수 버퍼 증인을 두지 않는다.

이 결과는 세 실제 생성 절차를 직접 호출하는 증명 작성 결합 모듈에 대한 정리다. `production_encode_hb_z1_full`과 `decode_hb_z1_full` 자체의 합성, 인코더/디코더 종료 또는 성공 분기 도달을 증명하지 않는다.

미해결 의무 6.2 (OBL-SIG-HBZ-ENCODE-DECODE). 이미 증명된 실제 `prepare/apply` 역함수와 실제 rANS 핵심 역함수를 production 전체 HBZ 래퍼 경계에서 성공 조건부로 합성한다. 래퍼의 반환 크기, 실패 분기, `encoder/copied/decoded` 배열 사이의 프레임과 실제 coefficient 결과를 함께 운반해야 한다. 별도로 **OBL-RANS-ACTUAL-SUCCESS-WITNESS**는 모든 기호가 6인 정준 입력에서 실제 성공 분기의 도달 가능성을 EasyCrypt 안에서 보여야 한다.

최신 판정은 GO-CORE이고 위 전체 HBZ 래퍼 합성이 Week 14의 단일 구현 간선이다. 이 경계가 컴파일되기 전에는 힌트(hint) h , 두 크기 차이, 정준 영 패딩과 전체 1474바이트 코덱으로 확장하지 않는다.

6.4 분리된 기능정확성 경로

OBL-SIGN-OUTPUT-TAIL-REACH는 적당한 Sign 출력이 Verify의 `unpack/norm` early-reject gate를 통과한다는 **PARTIAL**의무다. signature prefix inverse는 닫혔지만 전체 suffix와 이후 gate는 열려 있다. 이는 성공한 Verify에 대해 tail 도달을 증명한 검증된 정리 4.7의 역이나 귀결이 아니다. `full pack/unpack inverse, norm, hint, response` 및 산술 정확성이 필요하다. 이를 전체 challenge equality와 결합한 **OBL-SIGN-VERIFY-CORRECTNESS**는 현재 **BLOCKED**이다. 이 기능 경로를 accepted-forgery 보안 경로의 숨은 전제로 넣지 않는다.

6.5 그 다음 관문: 문맥(context)과 전체 도전값 전사(challenge transcript)

raw/internal 경로와 public-context 경로는 구현에서 실제로 다르다. Sign은 `_sf_prepare_pre_raw`와 `_sf_mu_preptr`를 사용하고, Verify는 `prelen`의 최상위 비트로 context branch를 고른다. 다음을 보여야 한다.

1. $ctxlen \leq 255$ 와 길이 인코딩의 정확성;
2. 양쪽이 정확히 $ctxlen\text{-byte} || ctx$ 를 같은 순서로 흡수함;
3. caller memory가 두 절차의 pointer/length 전제를 공급함.

그 다음 **OBL-HIGH-LSB-PACK**은 Sign과 Verify가 만드는 HighBits_h , LSB의 packed bytes가 같음을 보여야 한다. 이 결과를 기존 `challenge_input_eq_components`와 합성하면 전체 challenge 입력 리스트가 같아진다. 이후에야 FIPS202 domain/padding/squeeze와 binary challenge sampler를 포함한 실제 full call을 paper ROM query에 연결할 수 있다.

6.6 그 이후의 기능 및 확률적 의무

표 3: 현재 명명된 주요 미해결 의무

의무	성격	필요한 결과
OBL-API-KEY-MEMORY-RAW-ACCEPT	API/관계	승인된 Sign/Verify 추적 (accepted Sign/Verify trace)의 직접 저장 μ_{32} 등식; 프레임/국소성/제어는 이미 증명됨
OBL-DIRECT-OBSERVED-MU	API/관계	실제 순차 추적 사후상태(actual sequential-trace post-state)의 <code>observed_mu</code> 접두부 결론
OBL-MU-PRODUCT-REPLAY	의미론/ DEFERRED	생성 해시 반환값 (generated hash return)의 pRHL 정리를 완료된 추적 유령 필드(completed-trace ghost field)로 옮기는 정당한 규칙
OBL-SIG-PREFIX-FRAME	인코딩(encoding)	패킹 서명 꼬리(pack signature tail)와 도전값 출력 꼬리(challenge-output tail)의 보존; 언패킹 하위 꼬리(unpack low tail)는 증명됨
OBL-RANS-DECODE-REFINEMENT	인코딩(encoding); PROVED	정확한 실제 추적, 상태 필드와 mode-2 표 아래 실제 디코더의 $bad = 0$, 정확한 소비 크기(consumed size), 1024기호 일치(symbol equality)와 출력 꼬리 프레임(frame); 종료는 별도
OBL-RANS-ACTUAL-SUCCESS-WITNESS	인코딩(encoding); PARTIAL	모든 기호가 6인 입력의 실제 인코더가 종료하여 $bad = 0$ 을 반환한다는 증인; 성공 시 크기 범위는 그 뒤 기존 정리에서 따옴
OBL-RANS-CORE-INVERSE	인코딩(encoding); PROVED	실제 인코더 실패 분기를 보존하고, 성공 시 실제 복사 구간을 디코더의 정확 추적 입력으로 운반해 $bad = 0$, 정확한 소비, 1024기호 복원과 꼬리 프레임을 얻는 성공 조건부 부분정확성; 종료/성공 도달은 별도

의무	성격	필요한 결과
OBL-SIG-HBZ-ENCODE-DECODE	인코딩 (encoding); PARTIAL	prepare/apply 역함수와 실제 rANS 핵심 역함수를 production 전체 HBZ 래퍼에서 합성하고 프레임/크기 및 실제 성공 증인을 공급
OBL-SIG-H-ENCODE-DECODE	인코딩 (encoding); SPECIFIED	HBZ 부모(parent)가 닫힌 뒤 시작할 실제 힌트 코덱 역함수(hint codec inverse)
OBL-SIG-SUFFIX-METADATA, OBL-SIG-SUFFIX-PADDING	인코딩 (encoding)	크기 차이 왕복(size-delta round trip)과 1474바이트 정준 영 패딩(canonical zero padding)
OBL-SIG-FULL-CODEC-MODE2	인코딩 (encoding)	접두부(prefix)와 전체 접미부(suffix)를 합친 실제 패킹/언패킹 역함수(actual pack/unpack inverse), 실패 표시(bad flag)의 성공과 정준 구문 분석(canonical parse)
OBL-SIGN-OUTPUT-TAIL-REACH	기능정확성	정당한 Sign 출력의 전체 패킹/언패킹(full pack/unpack) 및 노름/힌트 관문(norm/hint gate) 통과
OBL-SIGN-VERIFY-CORRECTNESS	기능정확성	응답(response), 힌트(hint), 산술식과 전체 도전값을 포함한 정당한 서명의 승인(accept)
OBL-KG-TERMINATION, OBL-KG-DIST	확률/종료	재시도 반복문 (retry loop)의 무손실성(losslessness) 또는 종료, 실제 KeyGen 출력분포, 특이성 거부(singular rejection) 효과
OBL-FFT-TAIL , TieMin	수치해석	고정소수점 FFT 오차의 나쁜 사건 (bad event) 확률과 동률 최솟값(tied minimum) 정책의 차이
OBL-SIGN-ACCEPTED-DIST	확률분포	초구 표본기(hyperball sampler), 거부(rejection), 승인 출력(accepted output)이 논문 분포와 일치하거나 명시적 통계손실을 가짐
OBL-VERIFY-EVENT	대수/프로그램	Jasmin의 최종 승인 비트(accept bit)가 논문의 노름(norm), 힌트, 방정식, 도전값 조건과 동치
OBL-CHALLENGE	조합론/확률	도전값 지지집합(challenge support), 기수(cardinality), 점 확률(point probability), 최소 엔트로피(min-entropy)
OBL-FORK	암호 환원	두 새 전사(fresh transcript)에서 정확한 BST-MSIS 인스턴스(instance)를 추출
OBL-SIGROM-ADAPTER	게임 기반 보안	바이트 수준 공용 API를 정확한 Sig_ROM.Scheme 및 질의 계수(query accounting)에 인스턴스화

6.7 의존성에 따른 작업 순서

현재 코드와 가장 잘 맞는 순서는 다음과 같다.

1. 생성/재전달 경로는 현재 논문 경계로 유지하고 같은 호출자 이음매에 관찰 래퍼를 더하지 않는다.
2. Week 13에서 닫힌 실제 rANS 핵심 역함수를 prepare/apply 역함수와 production 전체 HBZ 래퍼에 합성한다. 세 하위 절차의 반복 불변식은 다시 증명하지 않는다.
3. 래퍼 수준 크기/프레임 및 실제 성공 증인을 공급해 **OBL-SIG-HBZ-ENCODE-DECODE**를 닫은 뒤 같은 rANS 불변식을 h 코덱에 재사용한다.
4. 크기 메타데이터(size metadata), 영 패딩(zero padding), 패킹/도전값 꼬리 프레임(pack/challenge tail frame)을 합성해 전체 1474바이트 코덱(codec)과 정준 구문 분석(canonical parse)을 닫는다.
5. 노름/힌트(norm/hint), 응답(response)과 산술 정확성을 결합하여 **OBL-SIGN-OUTPUT-TAIL-REACH**와 정당한 Sign/Verify 정확성 (legitimate Sign/Verify correctness)을 닫는다.
6. 별도의 보안 경로에서 공용 문맥(public-context) μ , highbits/LSB와 전체 도전값 호출(challenge call)을 합성한다.
7. KeyGen/Sign의 확률분포와 종료 손실을 정량화하고 바이트 충실 ROM 스킴 (byte-faithful ROM scheme)을 논문의 분포/포킹(distribution/forking) 의무와 결합한다.

이 순서는 넓은 보안 명제를 먼저 선언하고 하위 전제를 나중에 채우는 방식이 아니라, 이미 컴파일되는 가장 가까운 seam에서 바깥 경계로 한 단계씩 올라가는 방식이다.

7 신뢰 경계와 재현 방법

7.1 무엇을 신뢰하는가

이 증명 개발의 신뢰 기반(trusted computing base, TCB)은 다음을 포함한다.

- EasyCrypt 커널/형 검사기(kernel/type checker)와 가져온 표준 라이브러리;
- Why3 및 호출된 SMT 증명기(prover);
- Jasmin 구문분석기/컴파일러(parser/compiler)와 jasmin2ec 번역기;
- 해시(hash)로 고정된 Jasmin 소스, 생성 추출물, 재사용 EasyCrypt 산출물;
- 소스 해시(source hash) 및 LaTeX 빌드를 수행하는 호스트 도구.

jasmin2ec가 소스를 받아들이고 생성된 EasyCrypt가 컴파일된다는 사실은 번역기의 의미 보존 증명이 아니다. 별도의 검증된 번역기 정리가 없는 한 번역기 자체는 TCB에 남는다. 마찬가지로 메모리 안전성과 SCT/상수시간(constant-time)은 EUF-CMA에서 따라오지 않으며 별도의 트랙이다.

7.2 증명 검증 파이프라인

저장소 루트에서 다음을 실행한다.

```
./haetae-topdown-easycrypt/scripts/verify-all.sh
```

이 스크립트는 다음을 순서대로 검사한다.

1. 고정 소스(pinned source)와 읽기 전용 증명 트리(read-only proof tree)의 SHA-256 변동(drift);

2. 패커(packer), 서명 코덱, HBZ 준비/적용/전체 래퍼, API 키 복사 보조절차, 원시 API 호출자, 원시 μ , 전사 집중 추출(focused extraction)의 재생성과 hash 일치, Week 7 pack/unpack 과 Week 8 focused HBZ procedure body identity 및 Week 9 실제 rANS 결합 모듈(actual rANS harness)이 재사용하는 생성 대상 동일성(generated-target identity);
3. 실제 512단어 HBZ 기호 인증서(symbol certificate)의 결정적 재생성, 생성기/출력(generator/output) 해시 일치와 생성된 EasyCrypt 인증서 자체의 커널(kernel) 재검사;
4. Week 9 rANS 정리 표면에 더해 Week 10 배열/목록, 단어 단계, 내부 진행, 실제 내부 반복문, 직렬화와 성공 크기 정리 표면 및 Week 11 꼬리 인식 불변식, 생성 단어 단계, 최종 저장과 실제 인코더 추적 폐쇄, Week 12 디코더 커서/표/재생 불변식과 실제 추적 정제, Week 13 복사 추적/점별 입력/W64 크기 어댑터와 실제 핵심 역함수의 과대주장 검사;
5. 현재 매니페스트된 67개 작성 EasyCrypt 대상의 -no-eco 새 컴파일;
6. admit, sorry, 작성 axiom, 디버그/임시(debug/temporary) 선언 부재와 매니페스트(manifest) 완전성;
7. 선택된 상류 기준선(upstream baseline)의 새 컴파일(fresh compile);
8. 기존 주차별(sprint) LaTeX 연구 노트의 참조/인용 오류 없는 빌드.

추출 해시(hash)는 생성물 변동(drift)을 탐지하지만 번역 의미의 정당성을 대신하지 않는다. 또한 실행 요약은 매 실행 때 덮어쓰므로, 발표용 스냅샷에는 마지막 RESULT PASS 줄까지 포함된 완전한 로그를 보관해야 한다. WEEK13_REPORT.md가 기록한 Week 13 완료 실행의 종료 표식은 RESULT PASS authored-targets=67 cache=-no-eco다.

경계 명제 7.1 (67대상 완료 스냅샷). 2026-08-09의 Week 13 완료 실행은 Week 12의 65대상 기준선에 Mode2RansCoreCompositionBridge.ec와 Mode2RansCoreActualInverse.ec를 더한 67개를 모두 컴파일한 뒤 기준선, 주차별 LaTeX와 읽기 전용 소스 무변경 검사를 통과했다. 독립 검증 기록 logs/week13-independent-verifier.md도 최종 정리의 사전조건에 인코더 성공, 복사 추적, 디코더 성공이나 출력 등식이 숨어 있지 않음을 확인한다. 실행 중 생성되는 logs/verify-all-summary.txt는 마지막 줄이 RESULT PASS일 때만 완료 실행의 증거다. 그 줄이 없는 중간 또는 중단 실행은 보존된 완료 기준선을 갱신하지 않는다.

7.3 이 문서 빌드

이 안내서는 기존 sprint 노트와 독립된 XeLaTeX 문서다. 저장소 루트에서

```
sh haetae-topdown-easycrypt/theory-guide/build.sh
```

을 실행하면 haetae-topdown-easycrypt/theory-guide/main.pdf가 생성된다. 스크립트는 중단 오류와 정의되지 않은 참조/인용(undefined reference/citation)을 실패로 처리한다.

문서 작성 시 관찰된 도구 경계는 Jasmin/jasmin2ec 2026.03.0, EasyCrypt OPAM switch easycrypt-5.2, Why3 1.8.0이며, 구성된 prover에는 Alt-Ergo 2.6.0, CVC4 1.8.0, CVC5 1.2.1, Z3 4.12.6이 포함된다. 정확한 재현 결과는 버전 문자열보다 실행 로그를 우선한다.

완료 판정 기준

이 안내서가 컴파일되는 것과 EasyCrypt 정리가 컴파일되는 것은 서로 다른 증거다. 문서 배포 전에는 build.sh와 전체 verify-all.sh를 모두 성공시켜야 한다. 현재 증명 기준선은 67대상 전체 통과이며, 이 안내서의 별도 빌드도 함께 확인한다.

A 정리와 소스 인덱스

표 4: 본문의 핵심 선언을 찾기 위한 안정적 인덱스

선언 또는 주장	파일	읽을 때 확인할 내용
FC_KeyGen, FC_Sign, FC_Verify	easycrypt/specs/TopLevelGoals.ec	구현/논문 분포와 검증함수의 최상위 명세 등식
ImplementationSecurityComposition	easycrypt/specs/TopLevelGoals.ec	equation (1)의 추상 손실 예산
implementation_security_obligations	easycrypt/security/ImplementationSecurityGoals.ec	숨기지 않은 구현 보안 의무들의 conjunction
cryptographic_bounds_well_formed	easycrypt/security/SecurityAssumptions.ec	MLWE, BST-MSIS, ROM 가정 인터페이스
vk_prefix_eq	easycrypt/refinement/keygen/PackedKeyPrefix.ec	정의 3.1의 배열 수준 정의
pack_sk_m23_mode2_vk_prefix	easycrypt/refinement/keygen/ExtractedPackedKeyPrefix.ec	실제 생성된 packer의 992-byte prefix 부분정확성
keypair_internal_mode2_return_prefix	easycrypt/refinement/keygen/ExtractedPackedKeyPrefix.ec	실제 내부 mode-2 KeyGen 반환값으로의 lift
imported_mode2_full_first_attempt_equiv	easycrypt/refinement/keygen/ExistingFirstAttemptAdapter.ec	기존 first-attempt 정리의 전제를 강화하지 않는 import-only 연결
keygen_prefix_memory_relation_is_constructible	easycrypt/support/Mode2KeyMemoryBridge.ec	저장된 VK로부터 메모리 prefix를 구성하는 역사적 local witness
keygen_export_vk_mode2_prefix,	easycrypt/refinement/composition/	실제 KeyGen exporter의
keygen_export_sk_mode2_prefix	ApiKeyMemoryBridge.ec	992/1408-byte memory 접두부 부분정확성
sign_import_mode2_sk_prefix,	easycrypt/refinement/composition/	실제 Sign/Verify importer의 로컬
verify_import_mode2_vk_prefix	ApiKeyMemoryBridge.ec	992-byte 접두부 부분정확성
exported_mode2_regions_agree,	easycrypt/refinement/composition/	export/import 접두부에서 raw μ
imported_sign_sk_reaches_mu_memory	ApiKeyMemoryBridge.ec	memory 전제로 가는 순수 adapter; caller 합성은 별도
canonical_ui64_address_roundtrip,	easycrypt/refinement/composition/	raw 정수 주소와 W64 주소의
mode2_vk_region_bridge	RawApiAddressBridge.ec	canonical 왕복 및 구간 바인딩
keypair_raw_api_exact_export_trace	easycrypt/refinement/composition/RawApiKeygenExportComposition.ec	실제 raw KeyGen과 두 copy를 포함한 export trace 사이의 정확한 메모리/결과 동치
keypair_raw_api_exports_matching_prefixes	easycrypt/refinement/composition/RawApiKeygenSequentialExport.ec	실제 raw KeyGen 순차 export 뒤 외부 VK/SK 접두부 일치
sign_raw_api_exact_mu_trace,	easycrypt/refinement/composition/	실제 raw Sign과 trace의 정확한 관계
sign_raw_api_trace_binds_hash_inputs	RawApiCallerMuTrace.ec	및 hash 입력 관찰값
raw_api_region_reaches_mu_zero_loss	easycrypt/refinement/composition/RawApiMuReachability.ec	constructed store 없이 external prefix와 Sign import를 sk_memory_prefix로 연결

선언 또는 주장	파일	읽을 때 확인할 내용
verify_raw_api_exact_mu_trace, verify_cryptolab_exact_mu_trace	easycrypt/refinement/ composition/ RawApiVerifyMuTrace.ec	모든 early-reject/tail 분기에서 실제 Verify 결과와 memory를 보존하는 trace
verify_raw_api_actual_accept_ binds_hash_inputs raw_api_accepting_ execution_ hash_mu_zero_loss	easycrypt/refinement/ composition/ RawApiVerifyAcceptTrace.ec easycrypt/refinement/ composition/ RawApiAcceptedMu Composition.ec	실제 성공한 Verify의 tail 도달 및 VK/pre/message hash 입력 바인딩 accepted trace 사실로 실제 생성 hash-call μ prefix를 인스턴스화
raw_api_accepting_ execution_ generated_challenge_ suffix_zero_loss	easycrypt/refinement/ composition/ RawApiAcceptedMu Composition.ec	accepted trace 전제에서 위치-64 generated suffix helper 결과 일치
sign_output_copy_exact_ and_frames_reused, sign_raw_api_frames_ reused_regions sign_verify_generated_ raw_ mu_prefix_regionwise	easycrypt/refinement/ composition/ RawApiSignOutputFrame.ec easycrypt/refinement/ composition/ RegionLocalMuEquivalence. ec, RegionLocalMuTop.ec	실제 signature/siglen 출력과 actual raw Sign caller가 분리된 VK/SK/pre/message 구간을 보존 전체 memory 등식 없이 실제 생성 hash를 VK/pre/message read region별로 비교
raw_sign_then_verify_ actual_ exact_trace, sign_then_verify_accept_ binds_observed_inputs sign_verify_raw_mu_top_ wrappers_prefix	easycrypt/refinement/ composition/ RawApiDirectObservedMu.ec easycrypt/refinement/ composition/ ExtractedMuHashPrefix.ec	실제 Sign→Verify sequence와 trace의 결과/memory 동치 및 accepted descriptor 사후조건; stored μ 등식은 없음 생성 helper chain의 wrapper-level μ_{32} prefix
sf_mu_rawpre_refines_raw_top	easycrypt/refinement/ composition/ ExactMuTopControl.ec	실제 Sign raw top과 wrapper의 정확한 관계
verify_hash_mu_raw_ refines_raw_top	easycrypt/refinement/ composition/ ExactMuTopControl.ec	실제 Verify raw branch와 wrapper의 정확한 관계
sign_verify_generated_ raw_mu_prefix	easycrypt/refinement/ composition/ ExactMuTopControl.ec	실제 생성된 Sign/Verify 절차 사이의 μ_{32} prefix
sign_verify_mu32_absorb_ from_pos64	easycrypt/refinement/ composition/ ExtractedChallenge Absorb.ec	위치 64에서 실제 μ_{32} absorb helper의 결과 일치
generated_raw_mu_ preconditions_from_ keygen_prefix	easycrypt/refinement/ composition/ ExactMode2RawMu Composition.ec	KeyGen prefix에서 raw theorem 전제의 구체적 witness
generated_raw_mu_to_ challenge_suffix_zero_ loss	easycrypt/refinement/ composition/ ExactMode2RawMu Composition.ec	현재 raw generated-top 국소 seam의 zero-loss 합성 정리
challenge_input_eq_ components	easycrypt/refinement/ composition/ TranscriptBytes.ec	highbits, LSB, μ 가 각각 같을 때 전체 list transcript가 같다는 congruence

선언 또는 주장	파일	읽을 때 확인할 내용
canonical_challenge, canonical_signed_low, prefix_codec_ preconditions_ satisfiable	easycrypt/refinement/sign/ Mode2SignaturePrefixCodec. ec	prefix round trip의 canonical challenge/signed-low 전제와 all-zero 비공허성 witness
pack_sig_prefix_mode2_ layout, unpack_sig_prefix_mode2_ layout	easycrypt/refinement/sign/ Mode2SignaturePrefixPack. ec, Mode2SignaturePrefixUnpack. ec	실제 생성 prefix pack/unpack의 32-byte challenge 및 1024-byte low 배치와 unpack low-tail frame
pack_unpack_sig_prefix_ mode2_roundtrip	easycrypt/refinement/sign/ Mode2SignaturePrefix RoundTrip.ec	두 실제 생성 절차의 canonical mode-2 prefix round trip 부분정확성
canonical_hbz_mode2, canonical_hbz_mode2_ satisfiable	easycrypt/refinement/sign/ Mode2HbzCodecSpec.ec	mode-2 1024-계수 범위, leaf prefix/frame 술어와 all-zero canonical witness
encode_hb_z1_prepare_ mode2_correct, decode_hb_z1_apply_ mode2_correct	easycrypt/refinement/sign/ Mode2HbzPrepare.ec, Mode2HbzApply.ec	실제 prepare/apply leaf의 exact symbol mapping, bad word, coefficient 복원 및 bounded tail frame
encode_prepare_decode_ apply_mode2_inverse	easycrypt/refinement/sign/ Mode2HbzLeafRoundTrip.ec	두 actual leaf wrapper의 순차 합성으로 첫 1024개 HBZ coefficient 복원
pack_target_encode_hb_z1_ full_exact_focused, unpack_target_decode_hb_ z1_ full_exact_focused	easycrypt/refinement/sign/ Mode2HbzActualBoundary.ec	focused/full 및 signature pack/unpack 추출 경로가 같은 실제 HBZ full procedure를 노출함
mode2_hbz_table_ certificate, hbz_fast_step_matches_ math	easycrypt/refinement/sign/ Mode2HbzTableCertificate.ec	13-symbol interval/table field와 reciprocal quotient/overflow certificate
actual_mode2_hbz_ tables_certified	easycrypt/refinement/sign/ Mode2HbzSymbolWords Generated.ec	literal actual esyms/dsyms/symbol arrays가 table certificate를 만족한다는 재생성 가능 정리
pure_rans_step_inverse, hbz_fast_step_decode_ inverse	easycrypt/refinement/sign/ Mode2RansCore.ec	순수 quotient/slot inverse와 concrete reciprocal fast-step inverse; normalization byte 및 actual loop는 제외
renorm_bytes_readback, encode_trace_state_ bounds, valid_rans_trace_ canonical	easycrypt/refinement/sign/ Mode2RansByteStack.ec	0/1/2-byte normalization, <i>xs/cuts/bytes</i> trace, little-endian state와 canonical state-bound 정리
encoder_two_byte_ decoder_order, cursor_two_decrements_ no_underflow	easycrypt/refinement/sign/ Mode2RansNormalization.ec	actual W32 shift/truncate와 decoder shift/or의 정수 의미 및 W64 backward-cursor 안전성
copy_encoded_suffix_ correct	easycrypt/refinement/sign/ Mode2RansSuffixCopy.ec	실제 <code>__copy_encoded_suffix</code> 의 pointwise slice equality와 unused-tail frame
actual_rans_encode_ mode2_control, actual_rans_decode_ mode2_control	easycrypt/refinement/sign/ Mode2RansEncodeRefinement. ec, Mode2RansDecodeRefinement. ec	실제 생성 반복문(actual generated loop)의 구체적 mode-2 입력/제어 (concrete input/control)와 $bad \in \{0, 1\}$. 이 제어 정리와 별도로 인코더 추적 정제(encoder trace refinement)는 Week 11에, 디코더 추적 정제(decoder trace refinement) 는 Week 12에 각각 닫힘

선언 또는 주장	파일	읽을 때 확인할 내용
actual_rans_harness_ branches_on_encoder_ result	easycrypt/refinement/sign/ Mode2RansActualInverse.ec	실제 인코더-복사-디코더 (encoder-copy-decoder) 호출, 실제 결과와 실패/성공 분기(actual-result failure/success branch)와 디코더 입력 필드(decoder input field) 초기화; 기존 제어 정리 자체는 역함수(inverse)가 아니며 Week 13 의미 정리는 아래에 별도
symbol_list_of_array, symbol_suffix, encode_trace_suffix_ extension	easycrypt/refinement/sign/ Mode2RansArrayListBridge.ec	실제 배열의 앞 1024기호를 목록과 접미부로 연결하고 구간/프레임 연산 및 한 기호 추적 점화식을 증명
actual_mode2_encoder_ word_step_correct	easycrypt/refinement/sign/ Mode2RansEncoderWordStep.ec	실제 mode-2 표, W32/W64 역수 곱과 이동/편향 단어가 순수 빠른 인코더 단계와 같다는 단어 수준 정리
generated_mode2_encoder_ word_step_unfold, generated_outer_word_ update_matches	easycrypt/refinement/sign/ Mode2RansEncoderGenerated WordStep.ec	생성 표 필드 적재, 역수 상위 곱, 이동 사다리, 보수(complement), 편향과 실제 외부 단어 갱신을 기존 빠른 단계에 연결하는 검증된 Week 11 다리
normalization_len_cases, inner_progress_segment_ step	easycrypt/refinement/sign/ Mode2RansEncoder InnerProgress.ec	순수 정규화 길이 0/1/2, 반복 나눗셈과 정확한 낮은 바이트 접미부 및 한 단계 구간 보존
encoder_inner_phase_inv, encoder_inner_segment_ inv, encoder_inner_segment_ step	easycrypt/refinement/sign/ Mode2RansEncoderTrace.ec	실제 생성 내부 반복문에 쓰는 보수적 0.4 위상, 바이트 구간과 접두부 프레임 술어 및 실제 저장 단계
actual_w32_le_bytes_ serialize, actual_encoder_final_ state_ serialization	easycrypt/refinement/sign/ Mode2RansEncoder Serialization.ec	네 실제 패턴 set8의 리틀엔디언 직렬화, 외부/접두부 프레임과 성공 크기 산술 앞 정리
actual_store_w32_le_ prepend_trace_bytes	easycrypt/refinement/sign/ Mode2RansEncoder SerializationComposition.ec	직렬화된 최종 상태를 순수 꼬리 앞에 붙이고 초기 배열에 대한 구간/프레임을 보존하는 합성 정리; Week 11 최종화 정리에서 실제 네 저장과 함께 사용됨
actual_rans_encode_inner_ no_underflow, actual_rans_encode_ success_ size_bound	easycrypt/refinement/sign/ Mode2RansEncoder ActualInner.ec	실제 생성 _rans_encode의 내부 바이트/프레임 불변식과 실패 또는 안전 오프셋 사후조건; 성공 시 크기 4..1024의 직접 Hoare 부분정확성
encoder_inner_tail_inv, encoder_outer_tail_ advance_success	easycrypt/refinement/sign/ Mode2RansEncoderTail Invariant.ec	순수 꼬리를 함께 운반하는 내부/외부 불변식의 초기화, 실제 내부 진행, 정확한 0/1/2바이트 종료와 한 기호 전이를 제공하는 검증된 Week 11 층
encoder_outer_tail_ finalize_success	easycrypt/refinement/sign/ Mode2RansEncoder Finalization.ec	순수 최종 상태 직렬화와 누적 정규화 접미부를 결합하고 정확한 길이 및 접두부 프레임을 도출하는 최종화 정리
generated_encoder_outer_ finalize_success	easycrypt/refinement/sign/ Mode2RansEncoderGenerated Finalization.ec	생성된 오프셋 단어와 실제 네 리틀엔디언 저장을 순수 최종화 정리에 연결하는 검증된 다리
encoder_trace_post, encoder_generated_ success_ outer_post, actual_rans_encode_trace_ closure	easycrypt/refinement/sign/ Mode2RansEncoderActual TraceClosure.ec	실제 RansEncodeTarget.M. _rans_encode를 직접 열어 실패 또는 성공 시 정확한 전체 접미부, 길이, 오프셋과 프레임을 증명하는 Week 11 핵심 Hoare 정리

선언 또는 주장	파일	읽을 때 확인할 내용
actual_rans_encode_trace_refinement	easycrypt/refinement/sign/Mode2RansEncoderOuterRefinement.ec	핵심 폐쇄 정리를 실제 반환 튜플과 trace_bytes 식으로 펼친 성공 조건부 부분정확성 따름정리
decoder_state_at, decoder_cursor, decoder_cursor_final	easycrypt/refinement/sign/Mode2RansDecoderCursor.ec, Mode2RansDecoderCursorSteps.ec	순수 추적의 디코더 상태, 구간, 시작/한 단계/최종 커서와 각 구간 읽기가 전체 크기보다 앞선다는 Week 12 경계 정리
actual_mode2_decoder_word_update_correct, generated_decoder_symbol_expected	easycrypt/refinement/sign/Mode2RansDecoderWordStep.ec, Mode2RansDecoderActualWord.ec, Mode2RansDecoderGeneratedStep.ec	실제 symbol_words/dsyms_words 조화와 W32/W64 갱신을 수학적 디코더 단어 단계와 예상 기호에 연결
decoder_replay_guard_exact, decoder_inner_trace_step, decoder_outer_trace_exit_components	easycrypt/refinement/sign/Mode2RansDecoderNormalization.ec, Mode2RansDecoderActualTrace.ec	0/1/2바이트 부분 재생, 실제 내부/외부 반복 불변식, 정확한 소비 커서, $x = 2^{23}$, 출력 접두부와 꼬리 프레임
actual_rans_decode_trace_refinement	easycrypt/refinement/sign/Mode2RansDecoderTopHoare.ec	실제 생성 RansDecodeTarget.M._rans_decode를 직접 열어 정확 추적 입력에서 bad = 0, 정확한 소비 크기, 1024기호 복원과 출력 꼬리 보존을 증명하는 Week 12 Hoare 부분정확성
copied_suffix_is_exact_trace, segment_matches_implies_exact_decoder_segment_input, actual_decoder_input_from_configured_trace, encoder_success_size_word_bridge	easycrypt/refinement/sign/Mode2RansCoreCompositionBridge.ec	실제 인코더 구간과 복사 slice에서 디코더의 전역/점별 정확 추적 입력을 구성하고, 실제 상태 저장과 W64 크기 무렵을 연결하는 Week 13 어댑터
actual_rans_encode_copy_decode_inverse	easycrypt/refinement/sign/Mode2RansCoreActualInverse.ec	Mode2RansActualHarness.run 안에서 실제 인코더 실패 시 디코더 미실행/초기 상태 보존과 성공 시 실제 복사, 디코더 bad = 0, 정확한 소비, 1024기호 복원 및 꼬리 프레임을 합성하는 Week 13 성공 조건부 Hoare 부분정확성
CLAIM_LEDGER	CLAIM_LEDGER.md	현재 상태의 운영상 단일 기준(source of truth)
THEOREM_GRAPH	THEOREM_GRAPH.md	부모 보안 목표에서 현재 seam까지의 의존성
PRODUCT_REPLAY_DECISION	PRODUCT_REPLAY_DECISION.md	stored observed_mu transport를 동결한 의미론 근거와 거부한 접근
SIGNATURE_CODEC_OBLIGATIONS	SIGNATURE_CODEC_OBLIGATIONS.md	실제 1474-byte layout, prefix 증명과 suffix/rANS 의무
MODE2_HBZ_CODEC_OBLIGATIONS	MODE2_HBZ_CODEC_OBLIGATIONS.md	Week 8 HBZ leaf/table부터 Week 9 byte-stack/copy/harness까지의 상태와 success boundary
RANS_PROOF_DESIGN	RANS_PROOF_DESIGN.md	reverse normalization-byte stack과 actual loop refinement의 분해 및 거부한 monolithic 접근
RANS_BYTE_STACK_INVARIANT	RANS_BYTE_STACK_INVARIANT.md	canonical xs/cuts/bytes 의미, byte order, 두 intended actual loop invariant와 비공허성 경계

선언 또는 주장	파일	읽을 때 확인할 내용
RANS_ENCODER_INVARIANT	RANS_ENCODER_INVARIANT.md	Week 10 실제 인코더의 접미부 인덱스 관례와 Week 11 꼬리 인식 불변식, 생성 단어 갱신, 최종 직렬화 및 실제 전체 접미부 폐쇄
RANS_DECODER_INVARIANT	RANS_DECODER_INVARIANT.md	Week 12 참조 상태/커서, 외부 기호 접두부와 꼬리 프레임, 내부 0/1/2 바이트 재생 불변식, 초기 파싱과 실제 최종 검사 경계
WEEK7_REPORT	WEEK7_REPORT.md	FREEZE-A/GO-B 결정과 Week 8 직전 30-target 역사적 baseline
WEEK8_REPORT	WEEK8_REPORT.md	CONTINUE-RANS 판정, HBZ leaf/table/single-step과 Week 9 직전 38-target 역사적 경계
WEEK9_REPORT	WEEK9_REPORT.md	바이트 스택, 실제 접미부 복사와 결합 제어, 두 열린 외부 반복문 정제 및 44대상 역사적 경계
WEEK10_REPORT	WEEK10_REPORT.md	배열/목록 및 단어 연결, 실제 내부 바이트/프레임, 직렬화 앞, 성공 시 크기와 50대상 CONTINUE-ENCODER 경계
Week 11 encoder trace note	latex/sections/27-week11-rans-encoder-trace-closure.tex	꼬리 인식 불변식, 생성 단어 단계, 최종 직렬화와 실제 외부 접미부 폐쇄를 설명하고 남은 성공 증인/디코더 경계를 구분한 연구 노트
WEEK11_REPORT	WEEK11_REPORT.md	OBL-RANS-ENCODE-REFINEMENT의 성공 조건부 부분정확성 폐쇄, 57 대상 통과, GO-ENCODER 판정과 Week 12 단일 디코더 의무
Week 12 디코더 정제 메모 (decoder refinement note)	latex/sections/28-week12-rans-decoder-refinement.tex	실제 디코더의 정확 추적 입력, 커서/표/재생 불변식, 실제 최종 거절 검사와 남은 결합 간선을 설명한 연구 노트
WEEK12_REPORT	WEEK12_REPORT.md	OBL-RANS-DECODE-REFINEMENT의 정확 추적 부분정확성 폐쇄, 65 대상 통과, GO-DECODER 판정과 Week 13 단일 결합 의무
Week 13 rANS 핵심 합성 메모 (core composition note)	latex/sections/29-week13-rans-core-composition.tex	복사 추적, 점별 디코더 읽기, W64 크기 연결과 실제 결합 모듈의 실패/성공 의미 합성을 설명한 연구 노트
WEEK13_REPORT	WEEK13_REPORT.md	OBL-RANS-CORE-INVERSE의 성공 조건부 부분정확성 폐쇄, 67대상 통과, GO-CORE 판정과 Week 14 실제 전체 HBZ 래퍼 의무
Week 13 독립 검증 기록 (independent verifier)	logs/week13-independent-verifier.md	최종 정리의 실제 세 절차 호출, 비순환 사전조건, 실패/성공 사후조건과 67대상 완료 로그를 별도로 대조한 판정
Focused extraction manifest	manifests/focused-extraction.md	Jasmin root, 생성 파일, hash, 작성 정리 및 focused HBZ body identity의 대응
Generated certificate manifest	manifests/generated-certificates.sha256	512-word actual HBZ symbol certificate generator/output의 pinned hash

B 기호 및 약어

기능정확성(functional correctness, FC)

구현의 결과 또는 분포가 논문 명세와 일치한다는 목표.

원시/내부(raw/internal)

문맥 길이 인코딩을 쓰지 않는 내부 메시지 경로. Verify에서는 prelen의 값이 2^{63} 보다 작을 때 실제 원시 분기를 선택한다.

승인 경로(accepted path)

실제 Verify 반환값이 0인 실행. 현재 정리는 이 사건이 tail_reached를 함의함을 증명하지만, 임의 서명이나 정당한 Sign 출력이 승인 경로에 속한다고 주장하지 않는다.

정확 추적(exact trace)

실제 절차와 최종 메모리/반환값이 같은 작성 거울 프로그램(authored mirror). 유령(ghost) 관찰값은 실제 제어흐름이 해당 지점에 도달한 경우에만 의미가 있다.

구간 국소(region-local)

전체 메모리 등식 대신 절차가 실제 읽는 주소 구간의 바이트 등식만을 요구하는 관계. 다른 주소의 차이는 결과에 영향을 주지 않는다.

생성/재전달 경계(product/replay boundary)

생성 해시의 관계적 반환값 정리를 이미 완료된 순차 추적의 저장된 유령 결과로 운반하는 미해결 의미론 간선. 현재는 명시적으로 **DEFERRED**이다.

서명 접두부(signature prefix)

1474바이트 mode-2 서명의 첫 1056바이트. 32바이트 도전값 비트와 1024바이트 하위 계수를 담으며 접미부 메타데이터, rANS 적재량과 패딩은 포함하지 않는다.

HBZ

서명 접미부에 rANS로 압축되는 1024개의 하이비트 계수 층. mode-2 정준 범위는 $[-6, 7)$ 이고 준비 앞의 13기호 알파벳 0, ..., 12로 옮긴다.

rANS 단계(rANS step)

빈도 구간(frequency interval)을 이용해 상태와 기호를 가역적으로 갱신하는 산술 단계. 순수/역수 단일 단계와 이를 잇는 정규화 읽기/상태 범위 추적을 다루며, 별도의 Week 11/12 정리가 각각 실제 인코더와 디코더 외부 반복문을 연결한다.

정규화 바이트 스택(normalization-byte stack)

인코더가 기호를 역순으로 처리하며 뒤쪽 커서로 내보낸 바이트열을 디코더가 앞에서부터 소비하는 관계. 순수 $xs/cuts/bytes$ 의미와 단어 산술은 단혔고, 실제 인코더 내부 쓰기까지 포함한 누적 외부 접미부 연결은 Week 11에, 정확한 입력 아래 실제 디코더의 정방향 소비(forward consumption)는 Week 12에 단혔다. 두 정리를 실제 복사 호출과 순차 합성하는 간선은 Week 13에 단혔다.

배열/목록 연결(array/list bridge)

실제 배열의 앞 1024기호를 순수 목록과 그 접미부에 연결하고, 구간 일치와 프레임 연산을 제공하는 Week 10 층.

구간 일치(segment match)

배열의 지정 오프셋부터 순수 바이트 목록이 점별로 저장되어 있다는 술어 segment_matches.

접두부 프레임(prefix frame)

지정 오프셋보다 낮은 배열 바이트가 실행 전후에 바뀌지 않았다는 술어 `prefix_frame`.

실제 내부 반복문(actual inner loop)

생성된 `_rans_encode`의 정규화 반복문. Week 10의 보수적 0.4 위상 정리에 더해 Week 11 꼬리 인식 불변식은 실제 가드에서 정확한 정규화 길이 0/1/2를 도출하고 외부 누적 추적과 동일시한다.

꼬리 인식 불변식(tail-aware invariant)

현재 기호의 정규화 바이트뿐 아니라 이미 존재하는 순수 추적 꼬리까지 `segment_matches`에 함께 넣어 내부 반복 뒤 외부 한 기호 점화식을 복원하는 검증된 Week 11 불변식.

정확 추적 디코더 입력(exact-trace decoder input)

정준 1024기호에서 계산한 `trace_bytes`, 크기, 상태 필드, 실제 디코더 표와 모든 정규화 바이트 읽기를 하나의 함수적 표현 계약으로 묶는 Week 12 술어. 성공, 출력 등식이나 사후상태를 전제로 포함하지 않는다.

부분 재생 불변식(partial replay invariant)

한 디코더 기호 구간에서 이미 소비한 바이트 수 k , 커서와 축소 상태를 `decoder_replay_prefix`에 연결하는 내부 불변식. 실제 가드는 $k < |\text{segment}|$ 와 같고 구간 길이는 0, 1 또는 2다.

실제 rANS 결합(actual rANS harness)

실제 인코더 반환 `bad`가 0일 때만 실제 접미부 복사와 디코더를 실행하는 단항 합성. 기존 결합 정리는 분기와 디코더 입력 필드만 증명한다. Week 13의 별도 의미 정리는 복사된 구간을 정확 추적 입력으로 운반하여 실패 시 미실행과 성공 시 1024기호 왕복을 같은 결합 안에서 증명한다.

성공 조건부(success-conditioned)

정준 입력만으로 고정 버퍼의 인코딩 성공을 가정하지 않고, 실제 `size = 0` 실패 또는 0이 아닌 성공 결론을 명시하는 정리 형태.

생성 절차(generated procedure)

고정된 Jasmin 소스에서 `jasmin2ec`로 생성한 EasyCrypt 절차.

래퍼(wrapper)

증명 정렬을 위해 작성한 국소 모듈. 현재의 중심 정리는 래퍼가 아니라 실제 생성 절차를 결론 표면에 둔다.

무손실(zero loss)

해당 결정론적 이음매의 두 실행이 정확히 같은 결과를 내므로 별도의 통계 거리가 없다는 뜻. 전체 API 손실이 0이라는 뜻은 아니다.

도달 가능성 증인(reachability witness)

관계 정리의 전제가 모순이 아니고 앞선 절차의 반환값으로부터 구체적으로 구성됨을 보이는 정리.

TCB

신뢰 기반. 현재는 EasyCrypt/Why3/증명기와 Jasmin 번역기, 고정 소스 및 호스트 검증 도구를 포함한다.

현재 연구 전선

승인된 Sign/Verify 추적의 `observed_mu` 직접 비교는 필요한 전제가 부족해서가 아니라 pRHL 해시 반환값을 완료된 추적의 유령 필드로 옮기는 생성/재전달 규칙이 없어 논문 경계로 동결되었다. 기능 경로에서는 Week 11의 실제 인코더 성공 접미부 정제에

이어 Week 12가 실제 디코더의 초기 파싱, 표 조회, 0/1/2바이트 재생과 외부/내부 반복문을 직접 단았다. 정확한 `trace_bytes` 입력에서 종료한 실제 디코더는 `bad = 0`, 정확한 소비 크기, 1024기호 복원과 출력 꼬리 프레임을 만족한다. Week 13은 실제 인코더의 `segment_matches`와 복사 보조절차의 `slice_eq`를 그 점별 입력으로 운반해 실제 핵심 역함수를 성공 조건부 부분정확성으로 단았다. 67개 작성 대상 전체 검증이 통과했고 최신 판정은 GO-CORE다. 다음 단일 의무는 이 핵심 역함수와 `prepare/apply` 역함수를 `production` 전체 HBZ 래퍼에서 합성하는 것이다. 실제 성공 증인과 종료, `h` 역함수, 크기 메타데이터와 영 패딩도 남아 있다. 문맥, `highbits/LSB`, 확률분포와 게임 기반 보안은 여전히 별도 의무다.

참고문헌

- [1] CryptoLab. Existing HAETAE easycrypt proof artifacts, 2026. Local source roots reused only through premise-preserving import adapters.
- [2] CryptoLab. HAETAE specification, 2026. Pinned local artifact `CryptoLab_HAETAE.pdf`, 46 pages, produced 2026-02-04; SHA-256 `80d28b3af9af2a27dba0eb4746f9b5755e05cb9a86bb6596f45fda06c605f3f6`.
- [3] CryptoLab. Pinned HAETAE jasmin implementation tree, 2026. Local source root `haetae-ref-jasmin/` used for focused `jasmin2ec` extraction.